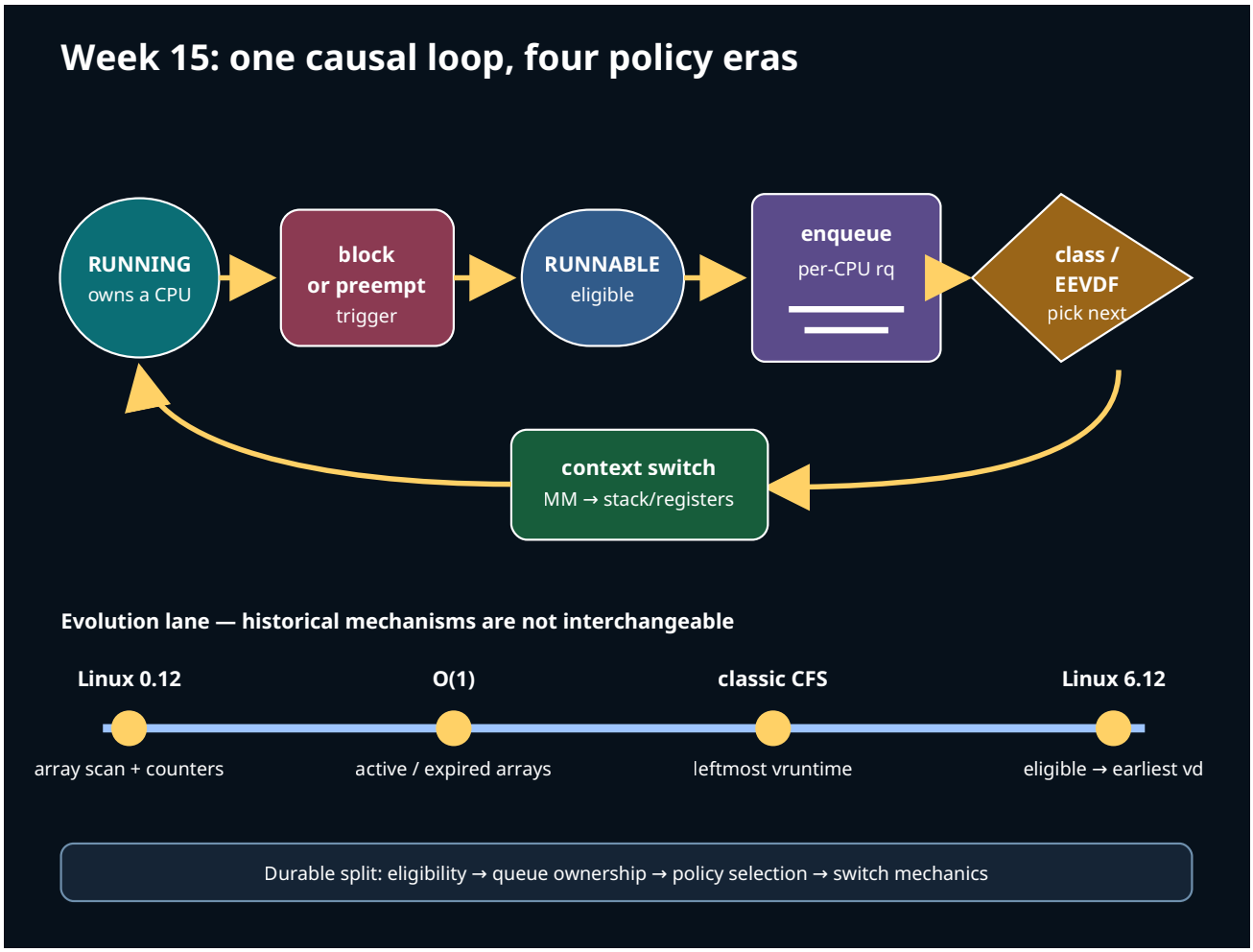
Week 15 — Linux Scheduler Evolution

Format: seven detailed mechanism lessons plus bounded userspace labs

Suggested workload: approximately 21 focused hours, adjustable to mastery

> WHY precedes HOW. Each day first states the scheduling problem, teaches only the C or assembly needed to read it, walks Linux 0.12, explains the pressure that forced evolution, marks the exact ToyOS boundary, reads pinned Linux 6.12, and finishes with a bounded observation whose limits are explicit.

The lab is projects/linux-scheduler. Modern source is pinned to Linux `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`. The source checker is read-only, matches every quoted modern landmark in normalized order, and hashes the source Makefile before and after. Current-upstream behavior beyond this pin is potentially changed unless independently version checked.



Required evidence discipline

Linux 0.12 is the compact causal model, from the locally supplied commented source PDF and early-source map. Its global task array, one-CPU assumptions, counters, 80386 TSS switching, and interrupt-masking conventions are historical. O(1) active/expired arrays and classic CFS's leftmost-vruntime explanation are later historical eras. Linux 6.12 fair selection uses EEVDF eligibility and virtual deadlines.

ToyOS contributes exactly what its source implements: two ring-0 tasks, `TASK_RUNNABLE/RUNNING/BLOCKED`, round-robin selection, dedicated stacks, a timer callback, and an x86-64 callee-saved-register `context_switch`. It has no userspace scheduler API, wait queues, per-CPU runqueues, SMP, EEVDF, scheduler classes, NUMA balancing, cgroups, or PI futex machinery.

Seven-day route

1. Day 1 — Scheduler Purpose, Causal Loop, and Architecture — wrapper day 1.
2. Day 2 — Blocking, Wait Queues, and Wakeups — wrapper day 2.
3. Day 3 — Per-CPU Runqueues and Context Switch — wrapper day 3.
4. Day 4 — Fair Scheduling: Counters to EEVDF — wrapper day 4.
5. Day 5 — Scheduler Classes and Guarantees — wrapper day 5.
6. Day 6 — SMP Placement, Topology, and Control — wrapper day 6.
7. Day 7 — Observation and Debugging Report — wrapper day 7.

Safety envelope

Core tests use only this process and children, honor the inherited cpuset mask, reap every child, arm timeouts, use no network, leave no temporary files, and do not mount tracefs, edit cgroups, request RT policy, offline CPUs, or create sustained pressure. Timing values are samples, never universal latency or switch-count claims. Optional tools print honest SKIPs.

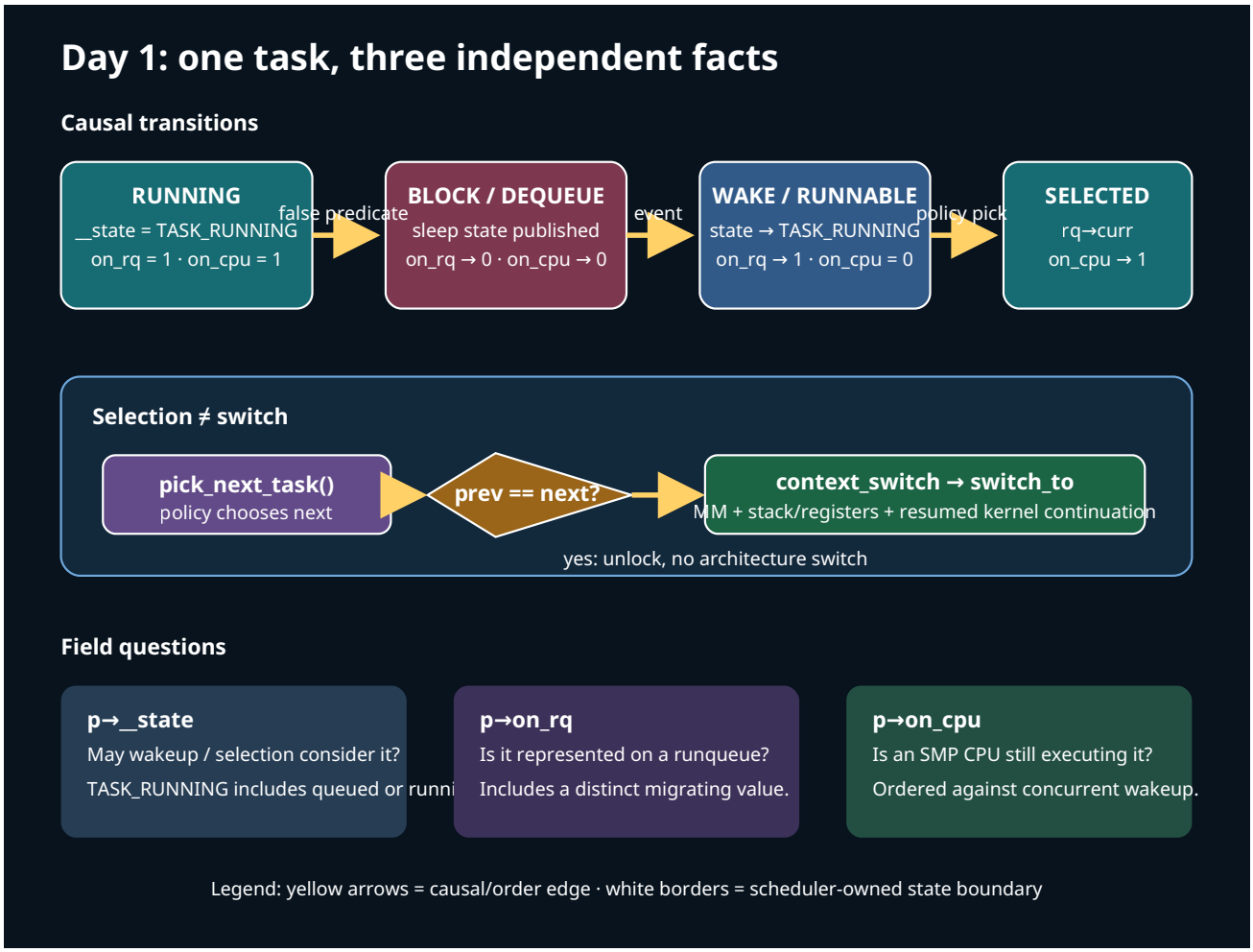
Completion checkpoint

For each day, submit the prediction, execution path, observation/source-proof split, limits, and completion prompt. A passing wrapper proves only that the bounded lab contract ran. It does not record mastery or alter learner state.

Day 1 — Why a Scheduler Exists: State, Eligibility, and the Switch

Target time: approximately 3 hours

> Driving question: what must the kernel know before it can give a CPU to a task, and which part of that decision is policy rather than mechanism?



Start with the pressure, not the algorithm

A CPU executes one instruction stream at a time, yet a system normally has more tasks than CPUs. Some tasks can make progress now; others are waiting for a pipe byte, a child, a lock, a timer, or storage. The scheduler exists to stop a waiting task from occupying a CPU and to divide finite CPU service among tasks that can run.

That gives the causal loop:

1. the current task runs until it blocks or scheduling is reconsidered;
2. blocking publishes a non-running state and removes the task from runnable competition;
3. the scheduler chooses among eligible queued tasks according to class policy;
4. the core and architecture switch execution context;

5. an external event can make a sleeper runnable again, but does not guarantee immediate execution.

Three facts are deliberately separate in modern Linux:

- `p->__state` describes sleep/wakeup eligibility. Despite the name, `TASK_RUNNING` means either runnable or currently running.
- `p->on_rq` records whether the task is represented on a runqueue (including a distinct migrating value).
- `p->on_cpu` records that an SMP CPU is still executing the task.

A task can be `TASK_RUNNING` and queued but not executing; it can be `TASK_RUNNING` and be `rq->curr`; during a switch, queue and CPU ownership transitions require ordering. Treating those fields as one “running bit” destroys the information a concurrent wakeup needs.

The architecture is similarly layered. A *trigger* (timer tick, wakeup, yield, block, or reschedule check) asks whether selection should be revisited. The scheduler *core* owns common state transitions, runqueue locking, accounting, and the call to switch. A scheduler *class* supplies policy callbacks such as enqueue, dequeue, pick, tick, and CPU selection. The architecture switch changes address-space and saved machine context. Returning to userspace is a later Week 14 boundary, not a synonym for context switching.

Minimum syntax needed to read the path

Read `p->sched_class->pick_next_task(rq, prev)` as a call through a function pointer: the core invokes a class-specific implementation while preserving one common call site. A macro such as `for_each_active_class(class)` is a generated loop, so its iteration order is part of scheduler policy.

`READ_ONCE(x)` and `WRITE_ONCE(x, value)` force a single compiler-visible access. They do not create an entire synchronization protocol. `smp_rmb()`, acquire/release operations, and spinlock boundaries express inter-CPU ordering; their comments identify the operation they pair with.

In assembly, `switch_to(prev, next, prev)` is not a normal function that merely returns a result. It changes the stack and register context, and execution eventually resumes at a continuation belonging to whichever task is switched back in. The x86-64 callee-saved registers (`%rbx`, `%rbp`, `%r12`–`%r15`) survive ordinary C calls; a kernel context switch also has to handle stack, instruction continuation, address space, and architecture-specific thread state.

Linux 0.12: a complete small causal path

Use `references/EARLY_LINUX_012_MAP.md`, Chapter 8.6 of the local commented Linux 0.12 PDF, printed pages 351–372. The value of this source is that the whole mechanism is visible together:

- `schedule()` checks alarms and signal effects, then scans the fixed `task[]` array.
- Only entries in `TASK_RUNNING` compete.
- The runnable task with the largest positive `counter` is selected.
- If all runnable counters are exhausted, every task's counter is recomputed from half its old value plus its static priority, and the scan repeats.
- `switch_to(next)` crosses the 80386 task-switch boundary.

The state test answers *may this task compete?* The counter answers *which eligible task wins this scan?* The switch answers *which saved execution context now owns the CPU?* Those are still useful distinctions even though none of the data structures should be projected onto current Linux.

The same chapter's sleep helpers put the current task behind a wait pointer, set a sleeping state, call `schedule()`, and later restore the older waiter. `wake_up()` changes the referenced waiter to `TASK_RUNNING`. This is enough to expose the circle “false condition → sleep → event → runnable → later selected.” It is not an SMP-safe modern wait queue: Linux 0.12 assumes one CPU, interrupt exclusion, fixed task slots, a single wait-pointer chain, a decaying counter heuristic, and hardware task switching.

Why the design had to evolve

The compact scan couples eligibility, ranking, and switching under assumptions that disappear as the machine grows.

Pressure	Required architectural change
More tasks	avoid a full fixed-array scan for every choice
Multiple CPUs	per-CPU runqueues, explicit ownership, migration, and lock ordering
Different service contracts	scheduler classes for stop, deadline, real-time, fair, and idle work
Interactive and weighted sharing	richer policy state than one decaying counter
Affinity, cpusets, topology, power, NUMA	selection of a legal CPU becomes distinct from ranking on that CPU
Concurrent sleep and wakeup	separately ordered state, `on_rq`, and `on_cpu` transitions
Virtualization and control groups	accounting, bandwidth, isolation, and placement constraints

Keep the eras precise. Linux 0.12 used array scans and counters. The later O(1) scheduler used active/expired priority arrays. Classic CFS ordered fair entities by virtual runtime. Pinned Linux 6.12's fair class uses EEVDF eligibility and virtual deadlines; that is not SCHED_DEADLINE, and it is not the old CFS picker. Across those changes, the durable split remains eligibility → policy choice → common switch mechanics.

The exact ToyOS boundary

The local source is narrower than Linux. In `projects/toyos/src/tasks.c`, `tasks[2]` holds two ring-0 kernel tasks with dedicated stacks. `schedule()` alternates IDs, avoids a `TASK_BLOCKED` task, validates the candidate stack and RIP, changes the old task to `TASK_RUNNABLE`, changes the next task to `TASK_RUNNING`, and calls `context_switch`. `scheduler_tick()` invokes that same scheduler when a current task exists.

`projects/toyos/src/switch.S` saves `%r15`, `%r14`, `%r13`, `%r12`, `%rbx`, `%rbp`, `%rsp`, and a local resume RIP into the old context, loads those values from the next context, and jumps to the next RIP. A later switch back reaches the local label and returns.

ToyOS therefore implements a two-task state filter, a fixed alternating policy, timer-triggered reconsideration, and an ABI-level kernel-stack switch. It has no user tasks, wait queues, wakeup API, SMP, `on_rq`/`on_cpu`, per-CPU runqueues, class hierarchy, affinity, EEVDF, PI, NUMA placement, cgroups, or Linux return-to-user path. `TASK_BLOCKED` is set only when a toy task finishes; it is not a reconstruction of Linux sleep/wakeup ordering.

Linux 6.12, pinned source

All excerpts below are verbatim from commit `adc218676eef25575469234709c2d87185ca223a` (tag `v6.12`). Line ranges refer to that checkout.

The task descriptor preserves separate facts rather than one overloaded state.

`include/linux/sched.h`, `struct task_struct`, lines 786–789

```
    unsigned int            __state;

    /* saved state for "spinlock sleepers" */
    unsigned int            saved_state;
```

`include/linux/sched.h`, `struct task_struct`, lines 807–824

```
#ifdef CONFIG_SMP
    int             on_cpu;
    struct __call_single_node   wake_entry;
    unsigned int            wakee_flips;
    unsigned long           wakee_flip_decay_ts;
    struct task_struct      *last_wakee;

    /*
     * recent_used_cpu is initially set as the last CPU used by a task
     * that wakes affine another task. Waker/wakee relationships can
     * push tasks around a CPU where each wakeup moves to the next one.
     * Tracking a recently used CPU allows a quick search for a recently
     * used CPU that may be idle.
     */
    int             recent_used_cpu;
    int             wake_cpu;
#endif
    int             on_rq;
```

In `__schedule()`, preemption is not treated as blocking. A nonzero sleep state on a non-preempt schedule path either loses to a pending signal or causes the task to leave runnable competition.

`kernel/sched/core.c`, `__schedule`, lines 6605–6620

```
    /*
     * We must load prev->state once (task_struct::state is volatile), such
     * that we form a control dependency vs deactivate_task() below.
     */
    prev_state = READ_ONCE(prev->__state);
    if (sched_mode == SM_IDLE) {
        /* SCX must consult the BPF scheduler to tell if rq is empty */
        if (!rq->nr_running && !scx_enabled()) {
            next = prev;
            goto picked;
        }
    } else if (!preempt && prev_state) {
        if (signal_pending_state(prev_state, prev)) {
            WRITE_ONCE(prev->__state, TASK_RUNNING);
        } else {
            int flags = DEQUEUE_NOCLOCK;
```

After state handling, the core asks policy for `next`. If `prev == next`, there is no architecture switch; entering the scheduling path does not imply a context switch.

`kernel/sched/core.c`, `__schedule`, lines 6647–6661

```
    next = pick_next_task(rq, prev, &rf);
picked:
    clear_tsk_need_resched(prev);
    clear_preempt_need_resched();
#ifdef CONFIG_SCHED_DEBUG
    rq->last_seen_need_resched_ns = 0;
#endif

    if (likely(prev != next)) {
        rq->nr_switches++;
        /*
         * RCU users of rcu_dereference(rq->curr) may not see
         * changes to task_struct made by pick_next_task().
         */
        RCU_INIT_POINTER(rq->curr, next);
```

The generic picker has a fair-only fast path above this excerpt; otherwise it walks active classes in priority order. Idle supplies the final runnable task, hence the terminal `BUG()`.

`kernel/sched/core.c`, `__pick_next_task`, lines 5984–6001

```
restart:
    prev_balance(rq, prev, rf);

    for_each_active_class(class) {
        if (class->pick_next_task) {
            p = class->pick_next_task(rq, prev);
            if (p)
                return p;
        } else {
            p = class->pick_task(rq);
            if (p) {
                put_prev_set_next_task(rq, prev, p);
                return p;
            }
        }
    }

    BUG(); /* The idle class should always have a runnable task. */
```

The switch boundary is wider than register restoration: `context_switch()` first handles MM/lazy-TLB state and lock handoff. These final lines expose the architecture transition and completion in the resumed context.

`kernel/sched/core.c`, `context_switch`, lines 5322–5331

```
    /* switch_mm_cid() requires the memory barriers above. */
    switch_mm_cid(rq, prev, next);

    prepare_lock_switch(rq, next, rf);

    /* Here we just switch the register state and the stack. */
    switch_to(prev, next, prev);
    barrier();

    return finish_task_switch(prev);
```

The resulting path is:

`kernel entry → trigger/reason → rq lock and state decision → dequeue if blocking → class policy picks next → rq->curr update → context_switch → architecture switch_to → resumed kernel continuation → eventual return checks`.

The core/class/architecture boundaries matter when debugging. A sleeper omitted from selection is a state or wakeup question. A runnable queued task that waits is a class, placement, bandwidth, or capacity question. A selected task that fails during transition is a switch/MM/architecture question.

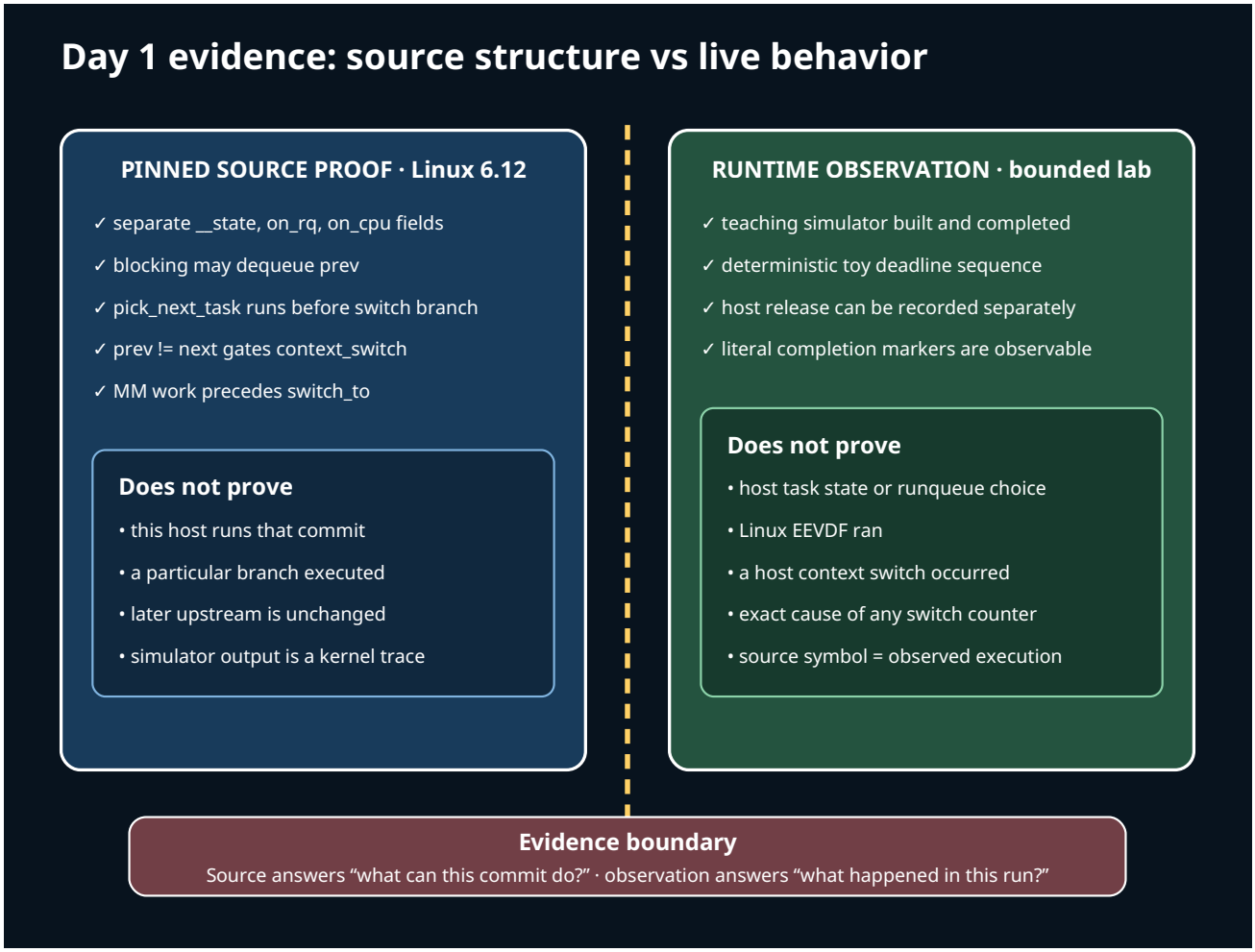
Bounded lab: observation is not implementation proof

Run the preserved wrapper:

```
15-linux-scheduler-evolution/challenges/day-001-purpose-task-state.sh
```

It builds through `projects/linux-scheduler/Makefile`, runs the pinned source checker, and executes `build/scheduler_sim`, then prints `PASS week15-day1`. The simulator has three synthetic entities and repeatedly picks the smallest toy `deadline`, producing a deterministic trace and `PASS scheduler_sim deterministic`.

This proves that the teaching program built, completed within its alarm, and followed its own deterministic selection rule. It does **not** run Linux EEVDF, inspect task state, select a host runqueue, exercise a scheduler class callback, or prove a host context switch. Its explicit `simulator=teaching_model_not_linux` marker is part of the contract, not a disclaimer to ignore.



Source reading proves that the quoted fields and control flow occur in this exact 6.12 commit. It does not prove the running host uses that source. Record `uname -r` separately. A source symbol cannot turn simulator output into a kernel trace, and a context-switch counter cannot identify the cause of any individual switch.

Reasoning checks and limits

Use these cases to test the model rather than memorizing names:

- **Runnable but not running:** verify state, `on_rq`, allowed CPU, class, throttling, and available capacity before calling it starvation.
- **Sleeping while a CPU is idle:** idle capacity cannot satisfy a false predicate or invent the missing event.

- **Scheduler entered but no switch:** `pick_next_task()` may return `prev`; reschedule flags and accounting can still be processed.
- **Switch occurred but no user return:** the next task resumes a kernel continuation. Week 14's exit-to-user work remains later.
- **Migration appears helpful:** elapsed time alone cannot separate capacity gain from cache and Week 13 NUMA-locality cost.

Connect the prior weeks explicitly. Week 10 task identity and lifetime survive selection. Week 11 synchronization creates valid sleep/wakeup edges; scheduling cannot repair a broken condition protocol. Week 12 pipes produce readiness events but userspace timing cannot name the kernel wake branch. Week 13 affinity and NUMA make legal placement different from memory locality. Week 14 entry, preemption legality, and return checks define where a reschedule request can actually be honored.

Finish by narrating one task around the entire causal loop. At each arrow label the evidence as `0.12 historical`, `ToyOS implemented`, `Linux 6.12 pinned`, `host observed`, or `hypothesis`. State one limit: current upstream beyond v6.12 is **potentially changed** until version-checked.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Purpose Task State

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

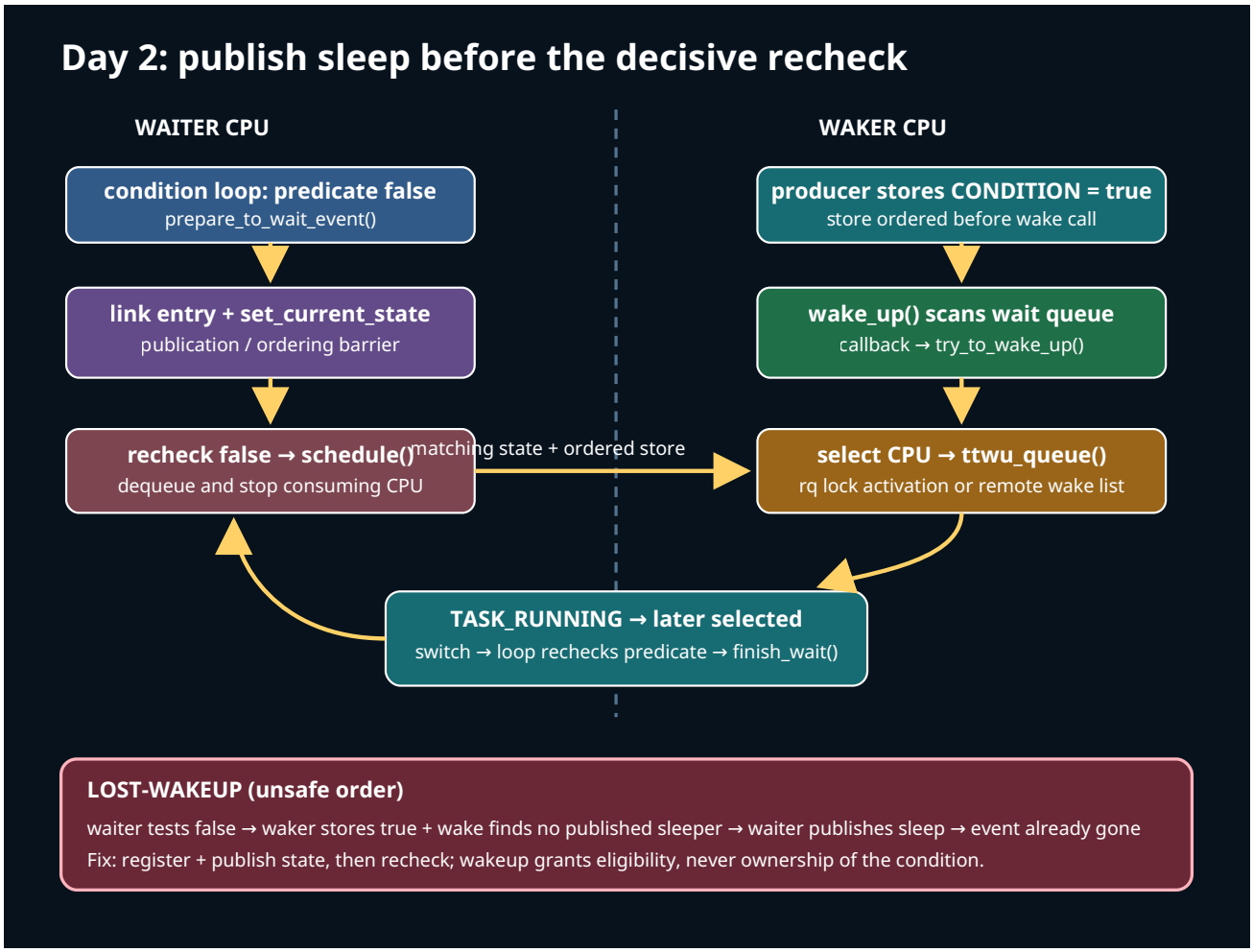
```
bash 15-linux-scheduler-evolution/challenges/day-001-purpose-task-state.sh
```

- View lab source on GitHub

Day 2 — Waiting Without Losing the Event

Target time: approximately 3 hours

> Driving question: how can a task stop consuming CPU while a condition is false without missing the change that makes the condition true?



Why waiting is a protocol

“Sleep, then wake” is an unsafe description. Suppose a reader sees an empty buffer, a writer fills it and sends a wakeup, and only then the reader publishes that it is asleep. The wakeup found no matching sleeper; the reader can now sleep while data is available. This is the lost-wakeup race.

The durable design is a condition loop:

1. protect or otherwise synchronize access to the condition;
2. register the waiter and publish a wakeable task state;
3. test the condition after publication;
4. call the scheduler only while the condition is still false;

5. after any wakeup, recheck the condition;
6. restore running state and remove the wait entry on every exit path.

The loop is necessary because a wakeup is permission to compete, not ownership of the condition. Another consumer may win before this task runs, signals may interrupt some waits, and wake functions can select candidates without guaranteeing that the predicate remains true.

A wait queue and the scheduler solve different parts. The wait queue records *who is interested in an event* and invokes wake functions under its lock. `try_to_wake_up()` decides whether a candidate's task-state mask matches, synchronizes against scheduling, and gets an unqueued task to a runqueue. Class policy later decides when that runnable task executes.

Minimum C and ordering syntax

Linux's `wait_event(wq, condition)` family is implemented as statement-expression macros: `({ ... })` behaves like an expression while containing declarations and a loop. A backslash continues a macro definition. `init_wait_entry()` creates a stack-local wait descriptor whose callback normally leads to `try_to_wake_up()`.

`list_empty()` asks whether a list node is linked; `list_del_init()` removes and reinitializes it. A wait entry is not the task state and neither one means “currently on a CPU.”

`set_current_state(state)` is stronger than a plain assignment: its ordering is designed to pair condition access with a waker. `__set_current_state()` lacks that full publication barrier and is used only where another synchronization rule suffices. `spin_lock_irqsave()` protects the wait-queue list while preserving interrupt state.

`state & mask` is a bit-mask match, not equality. A wakeup for `TASK_INTERRUPTIBLE` must not automatically activate a task sleeping in an unmatched state. `READ_ONCE`, `WRITE_ONCE`, `smp_rmb`, acquire loads, and spinlock barriers must be read as an ordered protocol; none is a magic global fence.

No new assembly is needed until the eventual switch. As on Day 1, `switch_to` resumes a saved kernel continuation. Waking a task changes eligibility and placement; it does not jump directly into the task's userspace instruction stream.

Linux 0.12: see the whole path, then stop

Read Chapter 8.6 (`kernel/sched.c`, printed pages 351–372) and the pipe path in `references/EARLY_LINUX_012_MAP.md` from the local commented PDF. The early sleep helpers place `current` behind a wait pointer, set a sleeping state, call `schedule()`, and arrange restoration of an older waiter. `wake_up()` changes the referenced task to `TASK_RUNNING`.

Chapter 12's early pipe reader makes the cause concrete. While the pipe is empty, it wakes writers that might now progress, checks for EOF and pending signals, and calls `interruptible_sleep_on()` on the read wait pointer. After bytes are consumed, readers wake blocked writers. The complete event chain—buffer condition, sleep, producer change, wake, retry—is compact enough to inspect end to end.

The stable lesson is that a resource condition owns the reason for waiting and its change owns the wake event. The implementation is historical: one CPU, interrupt exclusion, inode-embedded wait pointers, a waiter chain managed by sleep helpers, and no modern SMP memory-ordering or per-CPU runqueue transfer. Do not claim that its exact helper lines are equivalent to current wait queues.

Why modern Linux needed more machinery

One wait pointer cannot represent many waiters with different modes, exclusive wake-one behavior, poll keys, callbacks, signals, and cancellation. SMP adds the decisive race: the waker can run while the waiter is publishing state, leaving its runqueue, or still executing on another CPU. Per-CPU runqueues then make “make runnable” a question of ownership: which CPU is selected, who takes that `rq->lock`, and whether remote activation should be handed to the destination CPU.

The modern split answers those pressures:

- a wait-queue head owns a locked list of wait entries;
- `prepare_to_wait_event()` links the entry and publishes task state;
- the caller tests the real predicate in a loop and calls `schedule()`;
- `wake_up*()` scans candidates and invokes their wake functions;
- `try_to_wake_up()` serializes task state with `p->pi_lock`, then handles queued, switching, or sleeping cases;
- CPU selection chooses a legal target;
- `ttwu_queue()` either activates under the target `rq->lock` or uses a remote wake list;
- `finish_wait()` restores `TASK_RUNNING` and removes any remaining entry.

This complexity is not policy ornament. It closes races while avoiding unnecessary cross-CPU cache-line traffic.

The exact ToyOS boundary

`projects/toyos/src/tasks.c` has `TASK_RUNNABLE`, `TASK_RUNNING`, and `TASK_BLOCKED`, but names alone do not implement Linux waiting. Its two ring-0 kernel tasks are created runnable. A task sets itself blocked only in `task_finish()`, after its work is complete. `schedule()` skips a blocked peer and otherwise alternates tasks. There is no condition predicate, wait-queue head, wait entry, prepare/finish pair, wake function, signal-interruptible wait, `p->pi_lock`, runqueue lock, target-CPU selection, or remote wake list.

`projects/toyos/src/switch.S` can save one kernel context and load another, so ToyOS reconstructs the final relinquish/resume mechanism. It cannot demonstrate that a producer and waiter avoid a lost wakeup. Adding `TASK_BLOCKED` to a diagram does not fill that boundary.

Linux 6.12, pinned source

The following excerpts are verbatim from tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`. The line ranges are from that checkout.

The wait macro shows the condition loop in its real order: prepare first, test after publication, schedule through `cmd`, and always finish.

`include/linux/wait.h`, `___wait_event`, lines 299–321

```
#define ___wait_event(wq_head, condition, state, exclusive, ret, cmd)       \
({                                      \
    __label__ __out;                            \
    struct wait_queue_entry __wq_entry;                 \
    long __ret = ret;   /* explicit shadow */               \
                                        \
    init_wait_entry(&__wq_entry, exclusive ? WQ_FLAG_EXCLUSIVE : 0);    \
    for (;;) {                                  \
        long __int = prepare_to_wait_event(&wq_head, &__wq_entry, state);\
                                        \
        if (condition)                              \
            break;                              \
                                        \
        if (___wait_is_interruptible(state) && __int) {         \
            __ret = __int;                          \
            goto __out;                         \
        }                                   \
                                        \
        cmd;                                    \
    }                                       \
    finish_wait(&wq_head, &__wq_entry);                 \
__out:  __ret;                                      \
})
```

Preparation links the waiter while holding the wait-queue lock, then publishes the chosen state. The earlier signal branch in this function removes the entry while holding the same lock; the macro rechecks the condition before returning the interrupt result.

`kernel/sched/wait.c`, `prepare_to_wait_event`, lines 291–303

```
    } else {
        if (list_empty(&wq_entry->entry)) {
            if (wq_entry->flags & WQ_FLAG_EXCLUSIVE)
                __add_wait_queue_entry_tail(wq_head, wq_entry);
            else
                __add_wait_queue(wq_head, wq_entry);
        }
        set_current_state(state);
    }
    spin_unlock_irqrestore(&wq_head->lock, flags);

    return ret;
}
```

Cleanup first declares the current task running. It then uses a careful lockless emptiness check and takes the queue lock if removal is still necessary.

`kernel/sched/wait.c`, `finish_wait`, lines 356–379

```
void finish_wait(struct wait_queue_head *wq_head, struct wait_queue_entry *wq_entry)
{
    unsigned long flags;

    __set_current_state(TASK_RUNNING);
    /*
     * We can check for list emptiness outside the lock
     * IFF:
     *  - we use the "careful" check that verifies both
     *    the next and prev pointers, so that there cannot
     *    be any half-pending updates in progress on other
     *    CPU's that we haven't seen yet (and that might
     *    still change the stack area.
     * and
     *  - all other users take the lock (ie we can only
     *    have _one_ other CPU that looks at or modifies
     *    the list).
     */
    if (!list_empty_careful(&wq_entry->entry)) {
        spin_lock_irqsave(&wq_head->lock, flags);
        list_del_init(&wq_entry->entry);
        spin_unlock_irqrestore(&wq_head->lock, flags);
    }
}
```

`wake_up()` is a family of macros that reaches `__wake_up()`. The common scanner does not directly edit every task state: it calls each entry's wake function, honors callback results, and limits exclusive wakeups.

`kernel/sched/wait.c`, `__wake_up_common`, lines 73–97

```
static int __wake_up_common(struct wait_queue_head *wq_head, unsigned int mode,
            int nr_exclusive, int wake_flags, void *key)
{
    wait_queue_entry_t *curr, *next;

    lockdep_assert_held(&wq_head->lock);

    curr = list_first_entry(&wq_head->head, wait_queue_entry_t, entry);

    if (&curr->entry == &wq_head->head)
        return nr_exclusive;

    list_for_each_entry_safe_from(curr, next, &wq_head->head, entry) {
        unsigned flags = curr->flags;
        int ret;

        ret = curr->func(curr, mode, wake_flags, key);
        if (ret < 0)
            break;
        if (ret && (flags & WQ_FLAG_EXCLUSIVE) && !--nr_exclusive)
            break;
    }

    return nr_exclusive;
}
```

The default wake callback eventually enters `try_to_wake_up()`. For a task other than `current`, the condition store must become ordered before the task-state test. `p->pi_lock` serializes concurrent wakeups and stabilizes scheduling fields used for migration; its name does not mean every wakeup performs priority inheritance.

`kernel/sched/core.c`, `try_to_wake_up`, lines 4160–4170

```
     * If we are going to wake up a thread waiting for CONDITION we
     * need to ensure that CONDITION=1 done by the caller can not be
     * reordered with p->state check below. This pairs with smp_store_mb()
     * in set_current_state() that the waiting thread does.
     */
    scoped_guard (raw_spinlock_irqsave, &p->pi_lock) {
        smp_mb__after_spinlock();
        if (!ttwu_state_match(p, state, &success))
            break;

        trace_sched_waking(p);
```

The function reads state before `on_rq`, with explicit barriers and matching comments. If the task is already queued, `ttwu_runnable()` handles that case under its current runqueue lock. If it is no longer queued, the state becomes `TASK_WAKING` before `p->pi_lock` can be released for activation.

`kernel/sched/core.c`, `try_to_wake_up`, lines 4222–4230

```
        smp_acquire__after_ctrl_dep();

        /*
         * We're doing the wakeup (@success == 1), they did a dequeue (p->on_rq
         * == 0), which means we need to do an enqueue, change p->state to
         * TASK_WAKING such that we can unlock p->pi_lock before doing the
         * enqueue, such as ttwu_queue_wakelist().
         */
        WRITE_ONCE(p->__state, TASK_WAKING);
```

Remote activation can be transferred to the destination CPU. The wake-list path marks pending work and queues the task's wake entry; the destination later processes that list under its own runqueue lock.

`kernel/sched/core.c`, `__ttwu_queue_wakelist`, lines 3821–3829

```
static void __ttwu_queue_wakelist(struct task_struct *p, int cpu, int wake_flags)
{
    struct rq *rq = cpu_rq(cpu);

    p->sched_remote_wakeup = !!(wake_flags & WF_MIGRATED);

    WRITE_ONCE(rq->ttwu_pending, 1);
    __smp_call_single_queue(cpu, &p->wake_entry.llist);
}
```

`ttwu_queue()` exposes the ownership alternatives. A successful wake-list handoff returns without the waker taking the destination `rq->lock`; otherwise the waker locks that runqueue and activates the task there.

`kernel/sched/core.c`, `ttwu_queue`, lines 3943–3955

```
static void ttwu_queue(struct task_struct *p, int cpu, int wake_flags)
{
    struct rq *rq = cpu_rq(cpu);
    struct rq_flags rf;

    if (ttwu_queue_wakelist(p, cpu, wake_flags))
        return;

    rq_lock(rq, &rf);
    update_rq_clock(rq);
    ttwu_do_activate(rq, p, wake_flags, &rf);
    rq_unlock(rq, &rf);
}
```

When the old CPU has finished referencing an unqueued task, selection can choose a different legal CPU. Migration updates accounting and task CPU before the common queue step.

`kernel/sched/core.c`, `try_to_wake_up`, lines 4264–4287

```
        smp_cond_load_acquire(&p->on_cpu, !VAL);

        cpu = select_task_rq(p, p->wake_cpu, &wake_flags);
        if (task_cpu(p) != cpu) {
            if (p->in_iowait) {
                delayacct_blkio_end(p);
                atomic_dec(&task_rq(p)->nr_iowait);
            }

            wake_flags |= WF_MIGRATED;
            psi_ttwu_dequeue(p);
            set_task_cpu(p, cpu);
        }
#else
        cpu = task_cpu(p);
#endif /* CONFIG_SMP */

        ttwu_queue(p, cpu, wake_flags);
    }
out:
    if (success)
        ttwu_stat(p, task_cpu(p), wake_flags);

    return success;
```

The end-to-end causal chain is therefore:

`predicate false → wait entry linked + state published → predicate rechecked → schedule/dequeue → producer changes predicate → wake queue scans → state mask matches under p->pi_lock → queued/switching case resolved → CPU selected → target rq activation or remote wake-list handoff → TASK_RUNNING → later class selection → switch → predicate rechecked`.

The phrase “wake the task” compresses all of that. Use the expanded form whenever diagnosing a hang.

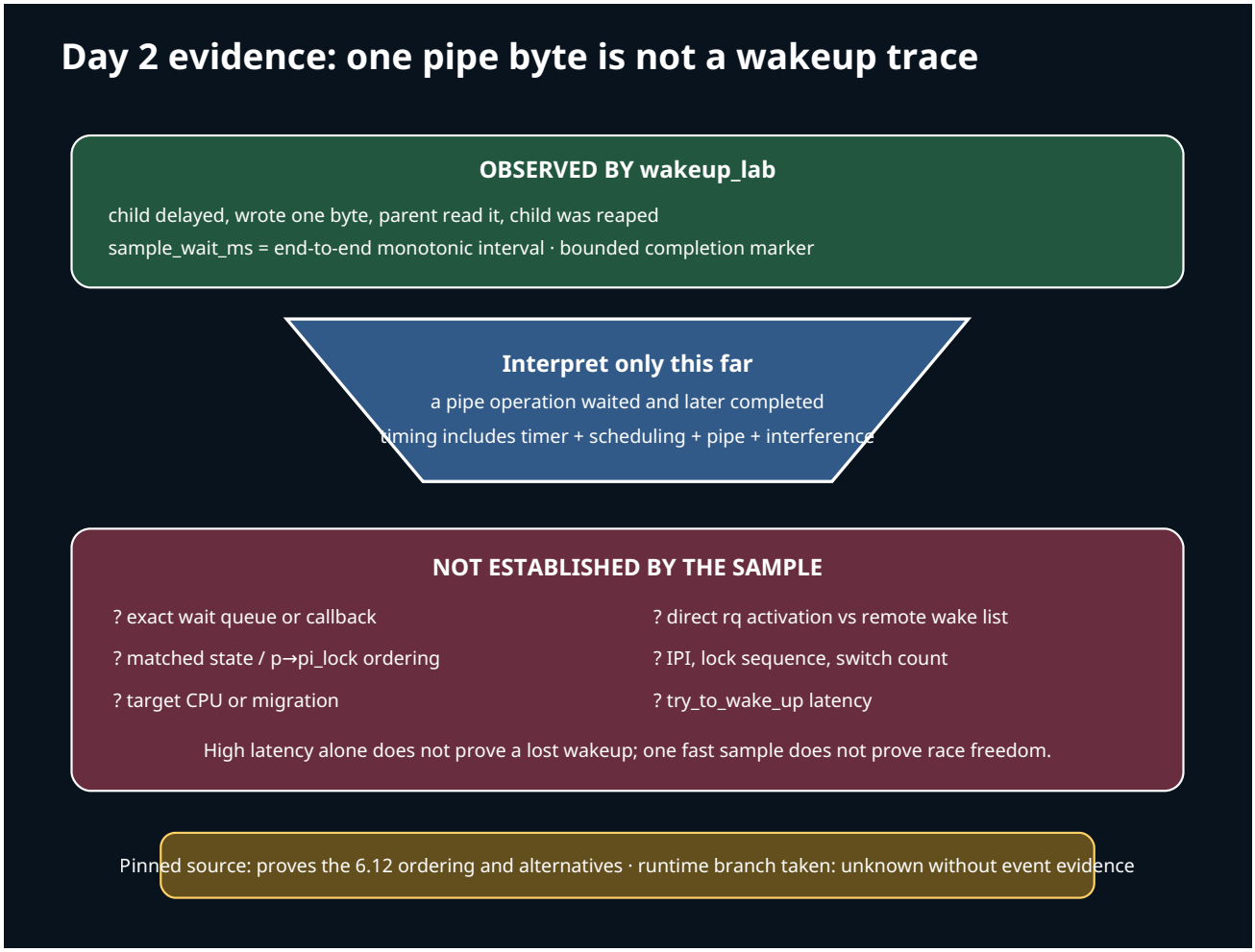
Bounded lab and evidence split

Run the preserved wrapper:

```
15-linux-scheduler-evolution/challenges/day-002-wait-wakeup.sh
```

It builds through `projects/linux-scheduler/Makefile`, runs the pinned source checker, executes `build/wakeup_lab`, and prints `PASS week15-day2`. The program creates a pipe, forks, lets the child sleep for about 20 ms and write one byte, blocks the parent in `read()`, reaps the child, and prints `sample_wait_ms=...` plus `claim=userspace_pipe_observation_only`. A four-second alarm bounds a stuck run.

That observation proves one byte crossed the pipe, the parent returned from `read`, the child was reaped successfully, and an end-to-end monotonic-clock interval was measured on the host. It does not prove which wait queue, wake callback, target CPU, `ttwu_queue` branch, IPI, lock ordering, or number of context switches occurred. The interval includes the child's timer delay, process scheduling, pipe implementation, clock calls, and host interference; it is not `try_to_wake_up()` latency.



The verbatim excerpts establish the ordering and alternatives in this pinned 6.12 source. They do not establish that `uname -r` identifies the same build, that the pipe run took every shown branch, or that later upstream is unchanged. Optional tracing can refine live evidence, but absence or permission denial must remain `SKIP`; do not mount tracefs or alter host-wide policy for this lab.

Lost-wakeup reasoning in practice

When a waiter hangs, ask in causal order:

1. **Predicate:** what exact resource fact is false, and under which lock or atomic protocol is it read?
2. **Publication:** was the wait entry linked and wakeable state published before the decisive recheck?
3. **Producer:** who changes the predicate, and is that change ordered before the wake call?
4. **Match:** does the wake mode match the published task state?
5. **Transfer:** was the task already queued, still `on_cpu`, activated on an `rq`, or handed to a remote wake list?
6. **Competition:** after becoming runnable, what class, affinity, bandwidth, or CPU-capacity constraint delayed execution?
7. **Loop:** did the resumed waiter recheck the predicate and clean up on every signal/error path?

Do not infer “lost wakeup” from high latency alone: the task may have become runnable and then waited for CPU service. Do not infer correctness from one fast sample: the dangerous interleaving may not have occurred. A task can also remain asleep correctly because the event did not match its state, or wake correctly and find that another consumer took the resource.

Weeks 10–14 supply the surrounding invariants. Week 10 keeps the task and referenced objects alive while it waits. Week 11 supplies predicate locks, atomics, condition loops, futex reasoning, and PI interactions. Week 12 supplies concrete descriptor readiness: pipe data, EOF, and backpressure cause wake events. Week 13 explains why the selected CPU and remote cache/NUMA cost matter after wakeup. Week 14 explains syscall/interrupt entry, legal scheduling points, preemption, and why a resumed kernel task still has a separate return-to-user path.

For completion, explain one pipe read from false predicate through recheck after wakeup. Label every claim as `Linux 0.12 historical`, `ToyOS boundary`, `Linux 6.12 pinned`, `host observation`, or `hypothesis`. End with the explicit limit: behavior after v6.12 is **potentially changed** until version-checked.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Wait Wakeup

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

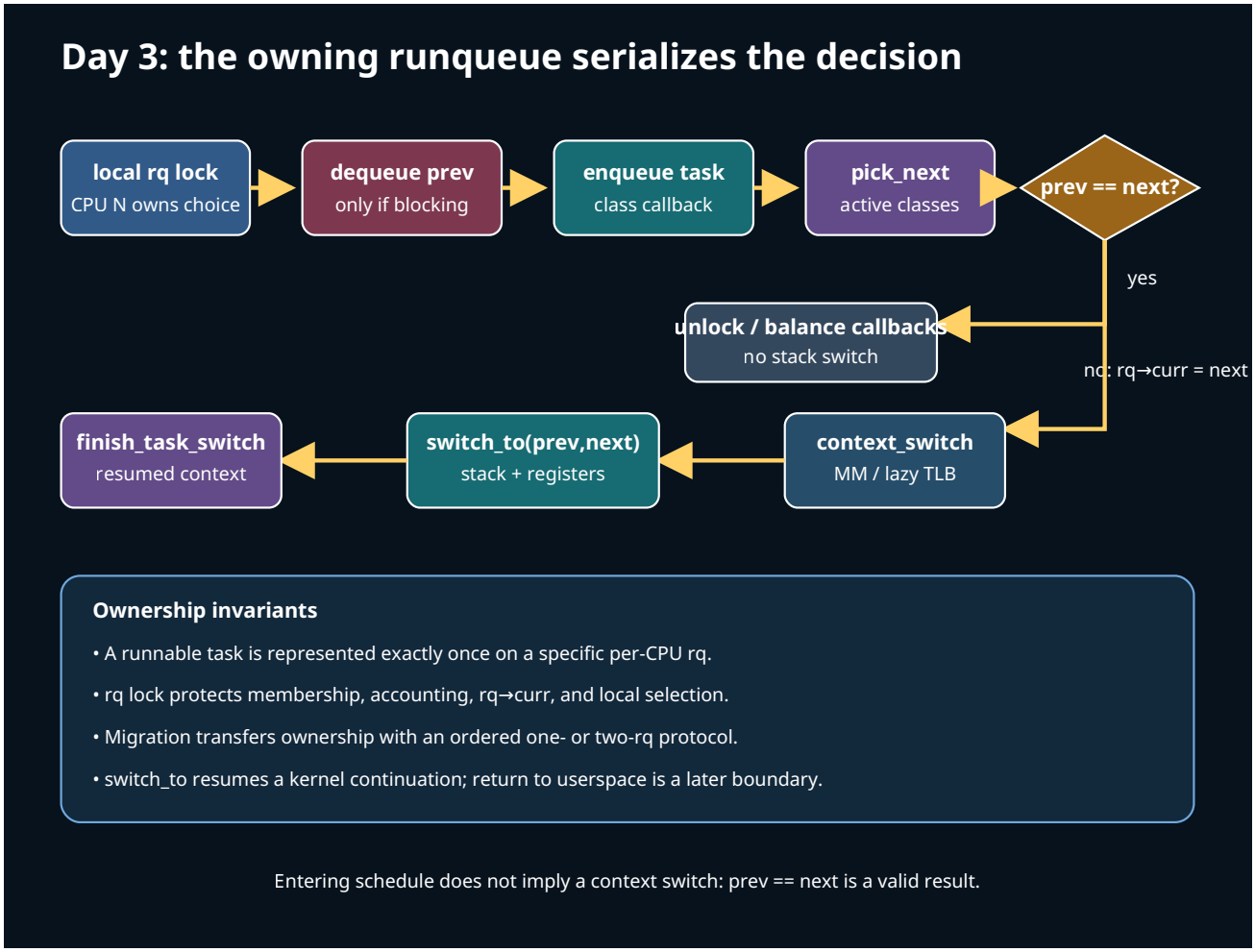
```
bash 15-linux-scheduler-evolution/challenges/day-002-wait-wakeup.sh
```

- View lab source on GitHub

Day 3 — Who Owns the Runnable Task?

Target time: approximately 3 hours

> Driving question: once a task is runnable, which CPU owns the decision to run it, and where does selection end and execution resume?



1. Syntax needed to read the path

Modern scheduler code is mostly ordinary C wrapped in conventions that carry locking and policy meaning.

- `this_rq()` and `cpu_rq(cpu)` return pointers to per-CPU `struct rq` objects. Per-CPU does not mean lock-free: queue membership and `rq->curr` are serialized by that runqueue's lock.
- `p->sched_class->enqueue_task(...)` is an indirect call through a function pointer. The core controls the transition; the selected class maintains its own queue.
- `READ_ONCE(prev->__state)` forces one compiler load. It does not replace the runqueue lock, the wakeup-side lock, or the paired ordering described in comments.
- `likely(prev != next)` is a branch-prediction hint, not a correctness condition.

- `switch_to(prev, next, prev)` is a macro. The third argument receives the task that was switched out when this saved continuation eventually resumes.

For assembly, read `%rdi` and `%rsi` as the first two x86-64 integer arguments. `pushq` saves on the current stack; changing `%rsp` changes which task's kernel stack is active. A label such as `1:` is a local resume point. Saving callee-saved registers is enough for ToyOS's C-call boundary, but not a complete Linux process switch.

2. Linux 0.12: one global decision is visible end to end

Start with Chapter 8.6, `kernel/sched.c`, printed pages 351–372 in the local commented Linux 0.12 PDF, using `references/EARLY_LINUX_012_MAP.md` as the reading guide. `schedule()` scans the fixed global `task[]` table for `TASK_RUNNING` entries and keeps the task with the largest positive `counter`. If no runnable task has positive credit, it recalculates every counter as half the old counter plus `priority`, then scans again. `switch_to(next)` invokes the 80386 hardware task-switch machinery.

That small implementation exposes the durable sequence:

```
state says runnable → policy ranks candidates → switch transfers execution
```

It does **not** expose modern ownership. There is one CPU, one global table, no remote enqueue, no pair of runqueue locks for migration, no `on_rq`/`on_cpu` handshake, and no separate scheduler classes. Its interrupt exclusion and hardware task switch are historical details, not a recipe for SMP Linux.

Check the old mechanism

Explain why a sleeping task with a large counter cannot win the scan. Then identify the point at which `counter` stops being merely accounting and becomes a selection key. Finally, name the first invariant that fails if two CPUs independently scan the same array and both select the same task.

3. Why evolution was necessary

With several CPUs, “runnable” is not enough. A task must be queued exactly once, on a specific CPU's runqueue, while wakeup, migration, blocking, and selection can race. A single global queue would concentrate lock traffic and destroy locality; an unlocked set of local queues could double-enqueue or lose a task.

Linux therefore needs explicit ownership:

1. A task's runqueue is selected subject to affinity and placement rules.
2. That runqueue's lock protects the membership transition.
3. enqueue/dequeue callbacks update both common accounting and class-private structures.
4. the owning CPU chooses from its own active classes;
5. migration transfers ownership under a defined one- or two-runqueue protocol.

Per-CPU queues trade one problem for another: they scale the common path, but now Linux must balance load, preserve CPU affinity, avoid deadlock when two queues are locked, and pay cache/NUMA costs when it moves execution. “CPU 3 is idle” does not imply a task may legally or profitably move there.

4. Exact ToyOS boundary

Read `projects/toyos/src/tasks.c:130-160`. Its private `schedule()` chooses one of two ring-0 kernel tasks by alternating IDs. It refuses a blocked candidate, marks the old task runnable, marks the new task running, and calls `context_switch()`. `scheduler_tick()` invokes the same function when a current task exists.

Read `projects/toyos/src/switch.S:1-25`. It stores `%r15`, `%r14`, `%r13`, `%r12`, `%rbx`, `%rbp`, `%rsp`, and a resume RIP in the old `struct context`, loads the same fields from the next context, and jumps. When the old task is later selected, it returns through the saved local label.

ToyOS therefore implements **choose → save continuation → load continuation → resume**, including timer-triggered and cooperative calls. It has one CPU, two statically known kernel tasks, no `mm`, no userspace, no runqueue lock, no migration, no class callbacks, and no Linux return-to-user work. Its state field is not evidence for Linux's `on_rq` or `on_cpu` protocol.

5. Linux 6.12, pinned: queue ownership and callbacks

All excerpts below are verbatim from commit `adc218676eef25575469234709c2d87185ca223a`. Line ranges identify this pinned tree.

`kernel/sched/core.c`, `enqueue_task()`, lines 2015–2034:

```
void enqueue_task(struct rq *rq, struct task_struct *p, int flags)
{
    if (!(flags & ENQUEUE_NOCLOCK))
        update_rq_clock(rq);

    p->sched_class->enqueue_task(rq, p, flags);
    /*
     * Must be after ->enqueue_task() because ENQUEUE_DELAYED can clear
     * ->sched_delayed.
     */
    uclamp_rq_inc(rq, p);

    if (!(flags & ENQUEUE_RESTORE)) {
        sched_info_enqueue(rq, p);
        psi_enqueue(p, flags & ENQUEUE_MIGRATED);
    }

    if (sched_core_enabled(rq))
        sched_core_enqueue(rq, p);
}
```

The caller supplies the already chosen `rq` and the required locking context. The class callback changes its queue first; common utilization, scheduler-info, PSI, and core-scheduling accounting surround it in a deliberate order.

`kernel/sched/core.c`, `dequeue_task()`, lines 2036–2058:

```
/*
 * Must only return false when DEQUEUE_SLEEP.
 */
inline bool dequeue_task(struct rq *rq, struct task_struct *p, int flags)
{
    if (sched_core_enabled(rq))
        sched_core_dequeue(rq, p, flags);

    if (!(flags & DEQUEUE_NOCLOCK))
        update_rq_clock(rq);

    if (!(flags & DEQUEUE_SAVE)) {
        sched_info_dequeue(rq, p);
        psi_dequeue(p, !(flags & DEQUEUE_SLEEP));
    }

    /*
     * Must be before ->dequeue_task() because ->dequeue_task() can 'fail'
     * and mark the task ->sched_delayed.
     */
    uclamp_rq_dec(rq, p);
    return p->sched_class->dequeue_task(rq, p, flags);
}
```

The boolean matters: delayed dequeue may leave an entity represented on the fair queue. Do not reduce the modern transition to “delete a tree node.”

6. Linux 6.12, pinned: select under the local lock

`__schedule()` identifies the current CPU, obtains that CPU's runqueue, disables local interrupts, and locks the queue. A blocking call differs from preemption: the former may remove `prev`; the latter leaves runnable work eligible.

`kernel/sched/core.c`, `__schedule()`, lines 6548–6567:

```
static void __sched notrace __schedule(int sched_mode)
{
    struct task_struct *prev, *next;
    /*
     * On PREEMPT_RT kernel, SM_RTLOCK_WAIT is noted
     * as a preemption by schedule_debug() and RCU.
     */
    bool preempt = sched_mode > SM_NONE;
    bool block = false;
    unsigned long *switch_count;
    unsigned long prev_state;
    struct rq_flags rf;
    struct rq *rq;
    int cpu;

    cpu = smp_processor_id();
    rq = cpu_rq(cpu);
    prev = rq->curr;

    schedule_debug(prev, preempt);
```

After the lock and state/order checks, the picker runs. If `prev == next`, Linux clears reschedule state and unlocks without switching stacks. Calling `schedule()` is therefore not proof that a context switch happened.

`kernel/sched/core.c`, `__schedule()`, lines 6647–6661:

```
    next = pick_next_task(rq, prev, &rf);
picked:
    clear_tsk_need_resched(prev);
    clear_preempt_need_resched();
#ifdef CONFIG_SCHED_DEBUG
    rq->last_seen_need_resched_ns = 0;
#endif

    if (likely(prev != next)) {
        rq->nr_switches++;
        /*
         * RCU users of rcu_dereference(rq->curr) may not see
         * changes to task_struct made by pick_next_task().
         */
        RCU_INIT_POINTER(rq->curr, next);
```

The omitted continuation performs accounting and tracing, then calls `context_switch(rq, prev, next, &rf)`. The queue remains part of a lock-handoff protocol; selection is not an unlocked query over a snapshot.

The generic picker normally walks active classes from highest to lowest. It has a fair-only fast path when all runnable tasks and the previous task satisfy its conditions. That optimization must return the same policy result; it is not a second scheduler.

7. The switch is three boundaries, not one

First comes address-space ownership, then architecture state, then continuation cleanup.

`kernel/sched/core.c`, `context_switch()`, lines 5322–5332:

```
    /* switch_mm_cid() requires the memory barriers above. */
    switch_mm_cid(rq, prev, next);

    prepare_lock_switch(rq, next, rf);

    /* Here we just switch the register state and the stack. */
    switch_to(prev, next, prev);
    barrier();

    return finish_task_switch(prev);
}
```

The preceding branch uses lazy-TLB handling for kernel threads or `switch_mm_irqs_off()` for a user address space. `switch_to` then changes stack and register state. The C call appears to return, but it returns when this task's saved continuation is selected, possibly much later.

`arch/x86/entry/entry_64.S`, `__switch_to_asm`, lines 177–191:

```
SYM_FUNC_START(__switch_to_asm)
    /*
     * Save callee-saved registers
     * This must match the order in inactive_task_frame
     */
    pushq   %rbp
    pushq   %rbx
    pushq   %r12
    pushq   %r13
    pushq   %r14
    pushq   %r15

    /* switch stack */
    movq    %rsp, TASK_threadsp(%rdi)
    movq    TASK_threadsp(%rsi), %rsp
```

Later lines restore the next task's callee-saved registers and jump to `__switch_to`, which handles additional architecture thread state. `finish_task_switch()` completes the lock handoff and deferred cleanup in the resumed context.

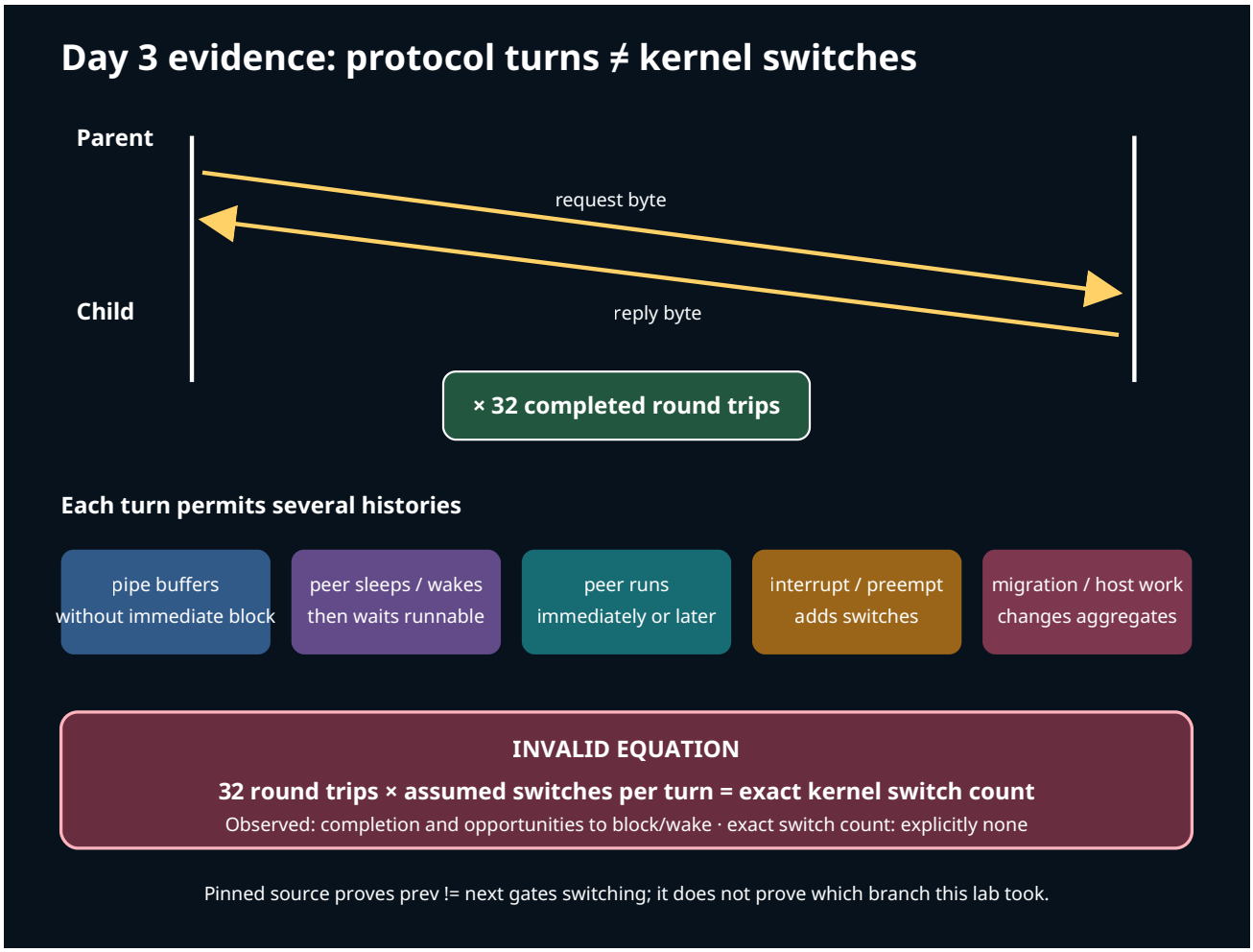
None of these operations is the return-to-user instruction. The chosen task first resumes a kernel continuation. It may continue a syscall, complete scheduling cleanup, or eventually reach Week 14's exit-to-user checks and only then execute the appropriate architecture return.

8. Bounded lab: what pipe ping-pong proves

Run:

```
./15-linux-scheduler-evolution/challenges/day-003-runqueue-switch.sh
```

The wrapper builds the existing `projects/linux-scheduler` binaries, verifies the pinned source anchors, and runs `switch_lab`. That program creates two pipes, forks, and performs 32 one-byte request/reply round trips under a four-second alarm.



Keep two evidence columns:

Observed from the lab

- the child was created and reaped successfully;
- all 32 protocol round trips completed;
- blocking pipe operations created opportunities for sleep, wakeup, and switching;
- the program explicitly reports `exact_kernel_switch_claim=none`.

Established only by pinned source

- queue operations dispatch through the task's scheduler class;
- the local runqueue and `rq->curr` participate in `__schedule()`;
- `prev != next` gates the architecture context-switch path;
- memory-context switching precedes `switch_to`.

The program cannot count exact kernel switches. Pipe buffering, wakeup timing, immediate execution of the peer, interrupts, involuntary preemption, and unrelated host activity all affect aggregate switch counters. Completion does not reveal which CPU owned either task, whether either migrated, which class picker branch ran, or whether this host runs the pinned 6.12 build.

9. Practical questions and limits

1. A task blocks after setting a sleep state. Under which runqueue lock is it removed, and what wakeup-side ordering prevents the event from being lost?
2. If two CPUs need to exchange tasks, why must their runqueue locks use an ordered protocol rather than “source then destination” on both CPUs?
3. If `prev == next`, which work in `__schedule()` still happened, and which switch counters must not be inferred?
4. When `next->mm == NULL`, why is copying ToyOS's register-only diagram insufficient?
5. If the ping-pong program completes faster after pinning both processes, does that establish fewer migrations, warmer caches, less host contention, or none of those without additional evidence?
6. Which event would you need to trace to distinguish enqueue on the waking CPU from enqueue on a remote target CPU?
7. Why can moving a task to an idle CPU improve capacity yet worsen cache or NUMA locality?

Stop at the version boundary: these excerpts prove implementation details only for the pinned Linux 6.12 commit. A host observation must record `uname -r`; a statement about later upstream is **potentially changed** until checked against a named commit.

Completion prompt

Narrate one blocking round trip as:

```
pipe condition → sleep state → dequeue → wake event → target rq enqueue
→ class pick → mm switch → register/stack switch → resumed kernel continuation
→ possible return to userspace
```

Label every arrow `Linux 0.12 historical`, `ToyOS implemented`, `Linux 6.12 pinned`, `host observed`, or `not established`. The answer is incomplete if `switch_to` and return-to-user are treated as the same boundary.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Runqueue Switch

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

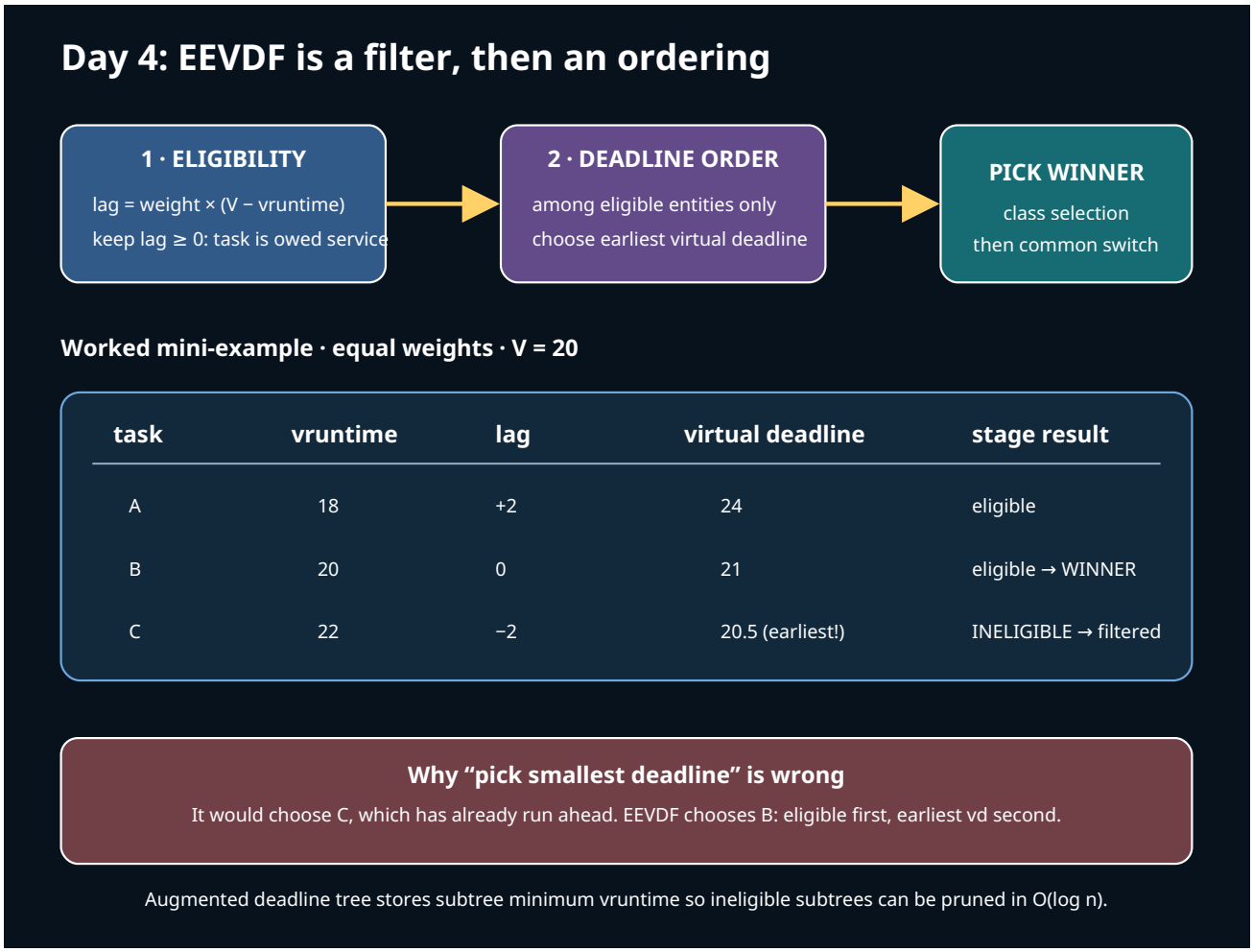
```
bash 15-linux-scheduler-evolution/challenges/day-003-runqueue-switch.sh
```

- View lab source on GitHub

Day 4 — Fair Service from Counters to EEVDF

Target time: approximately 3 hours

> Driving question: among runnable fair tasks, how did Linux move from remaining ticks to eligibility plus virtual deadlines?



1. Syntax and units before policy

Scheduler time fields are integers with policy-specific units. A `u64` subtraction often becomes an `s64` comparison so wraparound can be handled under bounded assumptions. Do not translate these expressions into wall-clock promises.

- `se->vruntime` is weighted service position in virtual time.
- `se->deadline` is an EEVDF **virtual** deadline, not a SCHED_DEADLINE absolute deadline and not a userspace completion time.
- `se->slice` is the current request length used to form a virtual deadline.
- `se->load.weight` changes how actual execution advances virtual time.

- `cfs_rq->avg_vruntime` and `avg_load` support an efficiently maintained weighted virtual-time estimate.
- `rb_node`, `rb_left`, and `rb_right` describe a red-black tree. Augmented fields summarize subtrees so a search can prune candidates without scanning every entity.

`calc_delta_fair(delta, se)` converts real-service-sized `delta` into weighted virtual time. A larger fair weight advances virtual time more slowly for the same execution. `entity_before(a, b)` and signed differences implement an ordering in this virtual domain; they are not ordinary unsigned "less than" tests.

2. Linux 0.12: counters, not EEVDF

Use Chapter 8.6, `kernel/sched.c`, printed pages 351–372 of the local commented Linux 0.12 PDF and `references/EARLY_LINUX_012_MAP.md`.

For each timer tick, the running task's positive `counter` is reduced. `schedule()` scans the fixed task table and picks a `TASK_RUNNING` task with the greatest counter. When all runnable counters reach zero, every task receives:

```
new counter = old counter / 2 + priority
```

The old remainder decays rather than disappearing, while static priority replenishes credit. This is a compact service heuristic: CPU-bound tasks consume counters; sleepers may return with credit. It has no explicit system virtual time, lag, request, eligibility test, virtual deadline, tree, or per-CPU fair runqueue.

Worked 0.12 example: runnable A and B have counters 6 and 2. A is selected and consumes ticks until another scheduling point changes the ranking. If both reach zero and their priorities are 3 and 5, they become 3 and 5. This does **not** mean B has an EEVDF deadline of 5; the number is replenished counter credit.

3. Why the fair scheduler evolved

The 0.12 scan costs linear work in the task-table size and assumes one CPU. More importantly, one decaying scalar cannot cleanly separate service entitlement, wakeup latency, request size, CPU placement, and bandwidth hierarchy.

Keep four eras separate:

1. **Linux 0.12 counters:** scan runnable tasks for greatest remaining credit; periodically recalculate counters.
2. **Historical O(1):** use priority-indexed active and expired arrays so choosing a priority does not scan all tasks. Interactivity and timeslice heuristics decide how tasks move through those arrays.
3. **Historical classic CFS:** charge weighted virtual runtime and choose the leftmost runnable entity from a vruntime-ordered red-black tree, approximating an ideal processor-sharing model.
4. **Pinned Linux 6.12 EEVDF:** test whether an entity is owed service, then choose the earliest virtual deadline among eligible entities. The fair tree is deadline ordered and augmented for eligibility search.

O(1) is not “early CFS,” and classic CFS's leftmost-vruntime explanation is not a full description of Linux 6.12. The historical designs explain why bounded pick cost, weighted service, sleeper behavior, and latency control were repeatedly revisited.

4. Exact ToyOS boundary

`projects/toyos/src/tasks.c:130-160` alternates between two runnable ring-0 tasks. It has no weight, service accounting, red-black tree, request length, lag, virtual time, or deadline. `projects/toyos/src/switch.S:1-25` changes saved continuations after that fixed choice.

ToyOS teaches that selection and context switching are separate mechanisms. It cannot answer which of two continuously runnable tasks is owed service. Any EEVDF values drawn beside its two task IDs are an explanatory overlay, not ToyOS state.

The Day 4 wrapper runs `projects/linux-scheduler/src/scheduler_sim.c`. That simulator contains three records and repeatedly chooses the smallest toy `deadline`, then increments it. Its own output says `teaching_model_not_linux`. It omits eligibility, vruntime, weights as Linux interprets them, augmented search, hierarchy, preemption, and SMP.

5. Lag and eligibility with numbers

EEVDF starts from service entitlement. In the source comment, lag for entity *i* is:

```
lag_i = service entitled - service received
      = weight_i × (V - v_i)
```

`V` is the runqueue's weighted average virtual time and `v_i` is the entity's vruntime. Non-negative lag means the entity has not run ahead of its entitlement and is eligible.

Example A: eligibility filters an attractive deadline

Assume equal weights and three queued entities:

```
entity    vruntime    lag at V=20    virtual deadline
A            18           +2               24
B            20            0               21
C            22           -2               20.5
```

C has the earliest deadline but has run ahead (`lag < 0`), so it is ineligible. A and B are eligible; B wins because deadline 21 precedes A's 24. “Pick the smallest deadline” alone would incorrectly choose C.

Example B: service changes eligibility

Two equal-weight entities start at vruntimes 10 and 14, so the simple average is 12. A has lag +2 and is eligible; B has lag -2 and is not. If A executes enough for its vruntime to reach 14, the average becomes 14 and both are at parity. Eligibility is not a permanent task property: accounting, enqueue/dequeue, and reweighting move the virtual time against which it is tested.

Linux avoids a division in the hot eligibility test. It compares weighted sums while including the current entity when it remains on the runqueue.

`kernel/sched/fair.c`, `vruntime_eligible() / entity_eligible()`, lines 726–745:

```
static int vruntime_eligible(struct cfs_rq *cfs_rq, u64 vruntime)
{
    struct sched_entity *curr = cfs_rq->curr;
    s64 avg = cfs_rq->avg_vruntime;
    long load = cfs_rq->avg_load;

    if (curr && curr->on_rq) {
        unsigned long weight = scale_load_down(curr->load.weight);

        avg += entity_key(cfs_rq, curr) * weight;
        load += weight;
    }

    return avg >= (s64)(vruntime - cfs_rq->min_vruntime) * load;
}

int entity_eligible(struct cfs_rq *cfs_rq, struct sched_entity *se)
{
    return vruntime_eligible(cfs_rq, se->vruntime);
}
```

The stable concept is “owed service.” The exact maintained sums and precision choices are pinned implementation details.

6. Virtual deadlines and request length

An eligible entity still needs a latency ordering. EEVDF associates a request with a virtual deadline:

```
virtual deadline = eligible virtual time + request / weight
```

In Linux's representation, `vruntime` is the virtual eligible-time basis and `calc_delta_fair(slice, se)` performs the weighted conversion.

`kernel/sched/fair.c`, `update_deadline()`, lines 1007–1029:

```
static bool update_deadline(struct cfs_rq *cfs_rq, struct sched_entity *se)
{
    if ((s64)(se->vruntime - se->deadline) < 0)
        return false;

    /*
     * For EEVDF the virtual time slope is determined by w_i (iow.
     * nice) while the request time r_i is determined by
     * sysctl_sched_base_slice.
     */
    if (!se->custom_slice)
        se->slice = sysctl_sched_base_slice;

    /*
     * EEVDF: vd_i = ve_i + r_i / w_i
     */
    se->deadline = se->vruntime + calc_delta_fair(se->slice, se);

    /*
     * The task has consumed its request, reschedule.
     */
    return true;
}
```

Worked request example in normalized equal-weight units: D and E are both eligible at virtual time 100. D requests 2 units and gets deadline 102; E requests 6 and gets 106. D is selected first. This improves latency for the shorter request, but D cannot keep submitting short requests to run indefinitely ahead: once its service makes it ineligible, deadline order cannot bypass the eligibility filter.

With unequal weights, the same real request produces a smaller virtual increment for the higher-weight entity. Nice therefore affects the virtual-time slope. A virtual deadline is still not a completion guarantee: higher scheduler classes, CPU availability, throttling, affinity, interrupts, and blocking all sit outside this local fair choice.

7. Why an augmented red-black tree is needed

Deadline order and eligibility order are different. The earliest-deadline node can be ineligible, as C was in Example A. Scanning deadline order until an eligible node appears could be linear. Linux stores subtree minimum vruntime so an entire subtree can be rejected when none of it can be eligible.

`kernel/sched/fair.c`, `pick_eevdf()`, lines 888–908:

```
/*
 * Earliest Eligible Virtual Deadline First
 *
 * In order to provide latency guarantees for different request sizes
 * EEVDF selects the best runnable task from two criteria:
 *
 *  1) the task must be eligible (must be owed service)
 *
 *  2) from those tasks that meet 1), we select the one
 *     with the earliest virtual deadline.
 *
 * We can do this in O(log n) time due to an augmented RB-tree. The
 * tree keeps the entries sorted on deadline, but also functions as a
 * heap based on the vruntime by keeping:
 *
 *  se->min_vruntime = min(se->vruntime, se->{left,right}->min_vruntime)
 *
 * Which allows tree pruning through eligibility.
 */
static struct sched_entity *pick_eevdf(struct cfs_rq *cfs_rq)
{
```

`kernel/sched/fair.c`, `pick_eevdf()`, lines 937–957:

```
    /* Heap search for the EEVD entity */
    while (node) {
        struct rb_node *left = node->rb_left;

        /*
         * Eligible entities in left subtree are always better
         * choices, since they have earlier deadlines.
         */
        if (left && vruntime_eligible(cfs_rq,
                    __node_2_se(left)->min_vruntime)) {
            node = left;
            continue;
        }

        se = __node_2_se(node);

        /*
         * The left subtree either is empty or has no eligible
         * entity, so check the current node since it is the one
         * with earliest deadline that might be eligible.
         */
```

The remaining search tests that node, otherwise moves right, and finally compares an eligible current entity. Red-black balancing bounds tree height; the augmentation makes eligibility pruning possible.

8. Where PELT and uclamp belong—and where they do not

PELT supplies decayed runnable/load/utilization signals. Those signals matter when Linux asks where work should run, how busy a CPU appears, how group load is represented, or what capacity/frequency response is appropriate. They are not the lag equation above.

Utilization clamping bounds utilization signals used by placement and frequency/capacity decisions. `uclamp_rq_inc()` appearing in the generic enqueue path does not mean uclamp sets `se->deadline`, grants more fair weight, or supplies a real-time reservation.

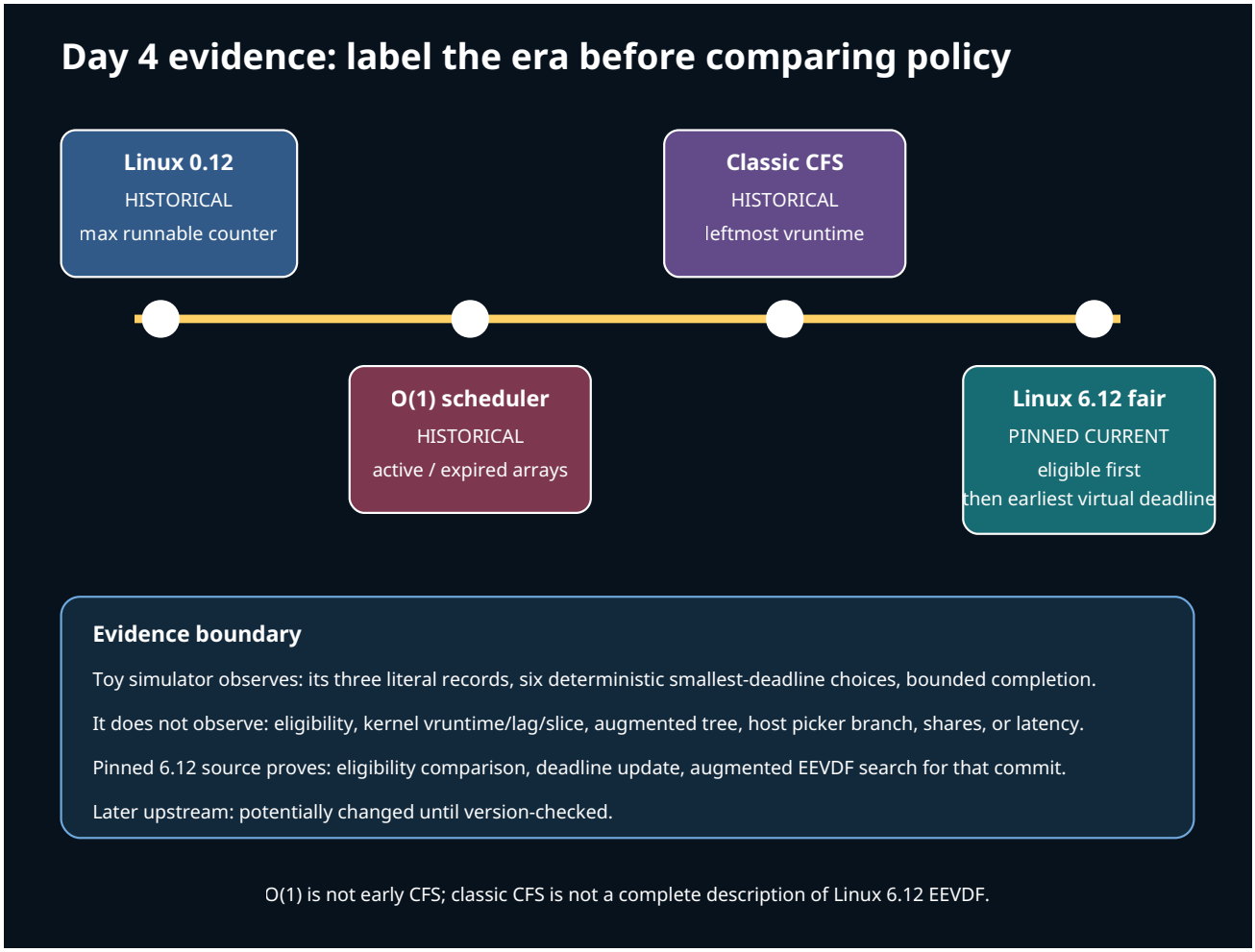
Only introduce PELT or uclamp in an EEVDF explanation when crossing from local fair selection to SMP placement or capacity. Otherwise they obscure the mechanism being studied.

9. Bounded lab and source-proof split

Run:

```
./15-linux-scheduler-evolution/challenges/day-004-fair-eevdf.sh
```

The wrapper builds the existing project, runs the source checker against the pinned commit, then executes `scheduler_sim`.



The run can prove

- the toy model starts from its three literal records;
- at each of six steps it chooses the smallest stored toy deadline;
- its deterministic update produces the printed sequence;
- the bounded program completes.

The run cannot prove

- that the host chose the process through `pick_eevdf()`;

- that any printed integer equals a kernel vruntime, lag, slice, or deadline;
- fair CPU shares, wakeup latency, tree shape, PELT values, or uclamp behavior;
- that the running kernel is Linux 6.12 at the pinned commit.

Pinned source can prove the eligibility comparison, deadline update, and augmented selection shown above for that commit. It cannot prove live host behavior.

10. Practical questions and limits

1. In Example A, why does deadline 20.5 lose to deadline 21?
2. If A's nice value changes, which quantities must be recomputed or interpreted with the new weight? Why is “nice is a deadline” false?
3. How can a task have the earliest deadline yet be ineligible?
4. Why would a tree ordered only by vruntime describe classic CFS better than pinned 6.12 EEVDF?
5. Which source field allows the picker to reject a left subtree without visiting every node?
6. A latency sample worsens after changing uclamp. What placement/frequency evidence is needed before blaming EEVDF selection?
7. How would cgroup hierarchy complicate a calculation that treats every task as a direct peer on one `cfs_rq`?

Fairness is conditional on available CPU time. A runnable fair task can wait behind stop, deadline, or RT work; a restricted affinity mask can remove capacity; a virtual machine can lose physical CPU time. One elapsed-time sample cannot separate those causes.

Completion prompt

Build one worked explanation with at least three entities. Compute eligibility first, then deadline order. Label the 0.12 counter, O(1) array, classic-CFS vruntime tree, and Linux 6.12 EEVDF statements by era. End with one statement that is `Linux 6.12 pinned` and one that is `later upstream potentially changed`.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Fair Eevdf

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

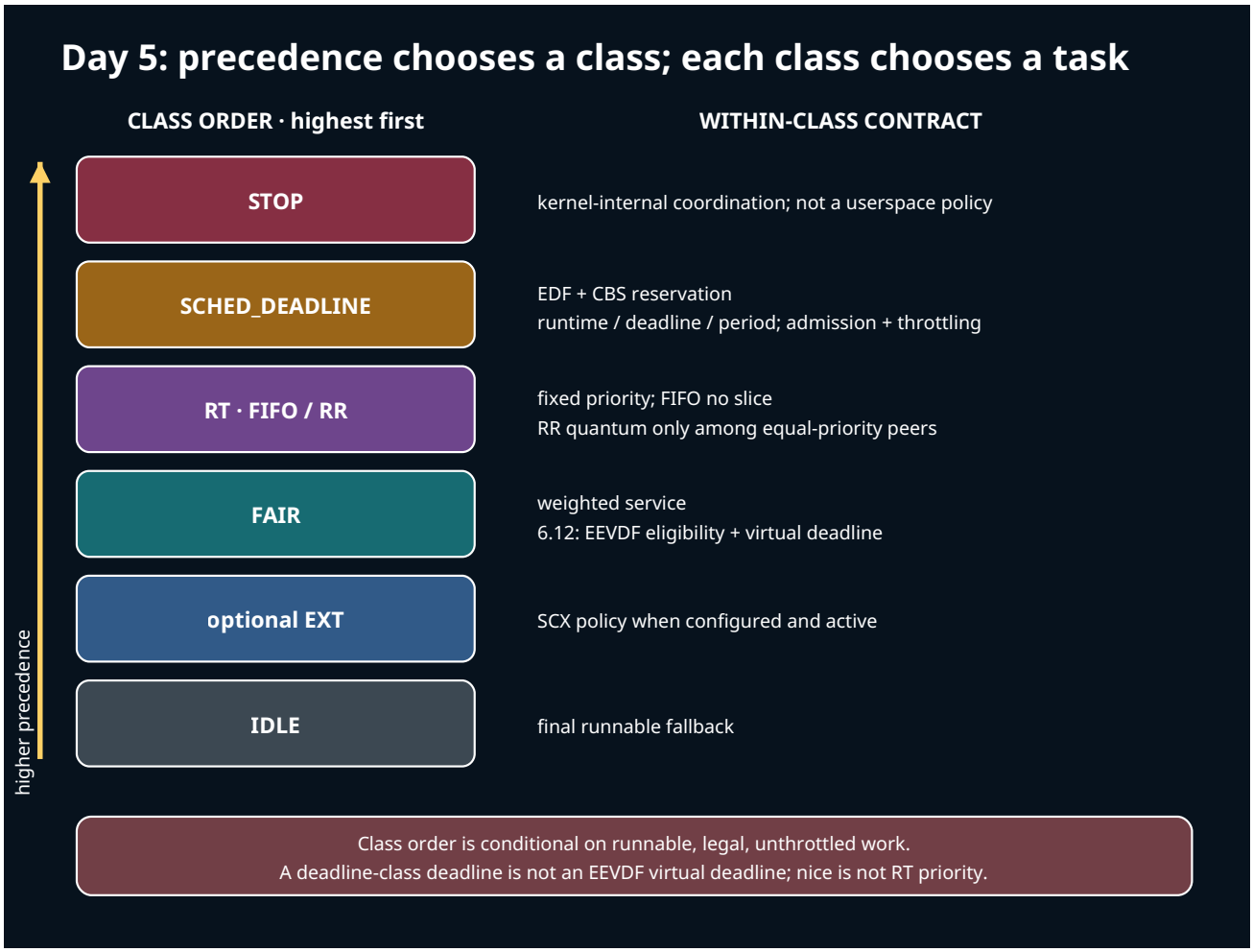
```
bash 15-linux-scheduler-evolution/challenges/day-004-fair-eevdf.sh
```

- View lab source on GitHub

Day 5 — Scheduler Classes, Contracts, and Priority Inversion

Target time: approximately 3 hours

> Driving question: what does Linux actually promise when runnable tasks belong to different policy classes, and what can break a priority argument?



1. Syntax for policy and precedence

A policy name, a priority number, and a scheduler class are related but not interchangeable.

- `p->policy` records a userspace-visible policy such as `SCHED_FIFO`, `SCHED_RR`, `SCHED_DEADLINE`, or a normal fair policy.
- `p->sched_class` points to the implementation operations used by the core scheduler.
- `const struct sched_class *` is a pointer to a read-only operations table.
- `DEFINE_SCHED_CLASS(rt)` places a class object into a linker section. The linker script, not a runtime sort, establishes class order.
- `for_each_active_class(class)` walks those section entries from highest to lowest, skipping optional classes when inactive.

- `sched_class_above(a, b)` compares class addresses in that ordered region. It is not a numeric-priority comparison.

Within the RT class, a higher userspace `sched_priority` outranks a lower one; kernel internal `prio` values use the opposite numeric direction. Nice values belong to the fair class. Keep each policy's namespace and rules separate. `SCHED_DEADLINE`'s `runtime`, `deadline`, and `period` are time parameters; an EEVDF virtual deadline is a fair-class implementation value.

2. Linux 0.12: one ranking rule, no class contracts

Read Chapter 8.6, `kernel/sched.c`, printed pages 351–372 in the local commented Linux 0.12 PDF, guided by `references/EARLY_LINUX_012_MAP.md`.

Linux 0.12 filters the fixed global task table by `TASK_RUNNING`, then selects the greatest positive `counter`. When all runnable counters are exhausted, it recomputes them from decayed credit plus static priority. `switch_to(next)` transfers execution.

This teaches the basic split between eligibility and ranking, but it cannot express:

- a stop task that must outrank ordinary work;
- a reserved runtime/deadline/period contract;
- FIFO versus round-robin behavior at a fixed RT priority;
- fair-service accounting below real-time work;
- an idle task selected only when nothing else can run;
- admission control, class bandwidth, throttling, or PI boosting.

Calling the old `priority` field an RT guarantee would be an anachronism. It contributes to counter replenishment in a uniprocessor heuristic.

3. Why policy classes became necessary

Different work asks for incompatible contracts. A fair desktop task wants proportional service and bounded interactive delay. A control task may need fixed-priority ordering. A deadline reservation describes runtime demand over a period. CPU-stop work is internal coordination that must not wait behind userspace. An idle thread must exist as a final fallback without competing as normal work.

One scalar ranking cannot state all those meanings. Linux splits common mechanics (state, runqueue ownership, enqueue/dequeue, accounting, and context switch) from class-specific selection. Precedence answers **which contract is considered first**; the chosen class then applies its own ordering.

The cost is that “highest priority wins” is incomplete. The answer depends on class, within-class priority or deadline, CPU affinity, admission, bandwidth state, throttling, blocking, and PI. A runnable fair task can starve while unbounded higher-class work remains runnable; throttling and reservations exist because precedence alone does not create CPU capacity.

4. Exact ToyOS boundary

`projects/toyos/src/tasks.c:130-160` alternates two ring-0 tasks, skips a blocked task, updates its three-state model, and invokes `context_switch()`. `projects/toyos/src/switch.S:1-25` saves and restores the minimal C continuation.

There is no policy attribute, class table, numeric RT priority, FIFO queue, RR quantum, deadline reservation, admission test, bandwidth timer, mutex PI, futex, userspace, or privilege check. ToyOS demonstrates that state filtering precedes a switch; it cannot demonstrate any Linux class guarantee.

The boundary matters especially here: a task that repeatedly yields in ToyOS is not `SCHED_RR`, and the timer-preempted worker is not `SCHED_FIFO`.

5. Linux 6.12, pinned: actual class precedence

All excerpts are verbatim from Linux commit `adc218676eef25575469234709c2d87185ca223a`.

The linker lays out the base classes in precedence order.

`include/asm-generic/vmlinux.lds.h`, `SCHED_DATA`, lines 129–138:

```
#define SCHED_DATA              \
    STRUCT_ALIGN();             \
    __sched_class_highest = .;      \
    *(__stop_sched_class)           \
    *(__dl_sched_class)         \
    *(__rt_sched_class)         \
    *(__fair_sched_class)           \
    *(__ext_sched_class)            \
    *(__idle_sched_class)           \
    __sched_class_lowest = .;
```

The required base precedence is:

```
stop > deadline > RT (FIFO/RR) > fair > idle
```

When `CONFIG_SCHED_CLASS_EXT` is present, the optional ext class is placed between fair and idle in this pinned layout and active-class iteration accounts for whether SCX is enabled. It does not move stop, deadline, or RT below normal fair work.

Initialization asserts the ordering rather than merely relying on documentation.

`kernel/sched/core.c`, `sched_init()`, lines 8366–8381:

```
void __init sched_init(void)
{
    unsigned long ptr = 0;
    int i;

    /* Make sure the linker didn't screw up */
#ifdef CONFIG_SMP
    BUG_ON(!sched_class_above(&stop_sched_class, &dl_sched_class));
#endif
    BUG_ON(!sched_class_above(&dl_sched_class, &rt_sched_class));
    BUG_ON(!sched_class_above(&rt_sched_class, &fair_sched_class));
    BUG_ON(!sched_class_above(&fair_sched_class, &idle_sched_class));
#ifdef CONFIG_SCHED_CLASS_EXT
    BUG_ON(!sched_class_above(&fair_sched_class, &ext_sched_class));
    BUG_ON(!sched_class_above(&ext_sched_class, &idle_sched_class));
#endif
```

The stop class is kernel-internal; there is no userspace `sched_setscheduler()` request that turns an application into a stop task. Idle is the final runnable fallback.

6. FIFO and RR: fixed-priority RT is conditional

`SCHED_FIFO` and `SCHED_RR` map to the RT class. A runnable task at a higher RT priority precedes lower RT priorities and lower classes on an eligible CPU.

FIFO has no scheduler timeslice. A selected FIFO task continues until it blocks, yields, is preempted by higher-priority work, leaves the policy, or is constrained by bandwidth mechanisms. Equal-priority FIFO queue-position rules matter at wakeup, yield, and policy changes; “runs forever” is not the contract.

RR uses the same RT priority ordering but adds a quantum among peers at the same priority.

`kernel/sched/rt.c`, `task_tick_rt()`, lines 2570–2589:

```
static void task_tick_rt(struct rq *rq, struct task_struct *p, int queued)
{
    struct sched_rt_entity *rt_se = &p->rt;

    update_curr_rt(rq);
    update_rt_rq_load_avg(rq_clock_pelt(rq), rq, 1);

    watchdog(rq, p);

    /*
     * RR tasks need a special form of time-slice management.
     * FIFO tasks have no timeslices.
     */
    if (p->policy != SCHED_RR)
        return;

    if (--p->rt.time_slice)
        return;

    p->rt.time_slice = sched_rr_timeslice;
```

RR does not make the RT class fair against fair-class tasks. A set of runnable RT tasks can prevent fair work from running unless they block or bandwidth controls intervene.

Pinned defaults illustrate RT bandwidth, but they are runtime configuration, not eternal constants.

`kernel/sched/rt.c`, file scope, lines 11–21:

```
/*
 * period over which we measure -rt task CPU usage in us.
 * default: 1s
 */
int sysctl_sched_rt_period = 1000000;

/*
 * part of the period that we allow rt tasks to run in us.
 * default: 0.95s
 */
int sysctl_sched_rt_runtime = 950000;
```

Throttling protects capacity for other work under the configured RT bandwidth regime. Administrators can change settings, cgroup hierarchy can add constraints, and boosted work receives special treatment. Never infer the active values from this source excerpt; inspect the running system with appropriate permissions.

7. SCHED_DEADLINE: reservation, admission, and throttling

Deadline class outranks RT in the class walk, but its contract is not “the earliest timestamp always runs regardless of demand.” A task supplies runtime, relative deadline, and period. Admission control limits accepted utilization; CBS accounting limits each reservation's interference.

`kernel/sched/deadline.c`, file header, lines 2–12:

```
/*
 * Deadline Scheduling Class (SCHED_DEADLINE)
 *
 * Earliest Deadline First (EDF) + Constant Bandwidth Server (CBS).
 *
 * Tasks that periodically executes their instances for less than their
 * runtime won't miss any of their deadlines.
 * Tasks that are not periodic or sporadic or that tries to execute more
 * than their reserved bandwidth will be slowed down (and may potentially
 * miss some of their deadlines), and won't affect any other task.
 *
```

The source's guarantee is conditional: the workload model, accepted reservation, and runtime demand matter. Exhausted runtime is replenished on the reservation's period and the entity can be throttled; overload, affinity constraints, blocking, hardware failure, and incorrect demand estimates are not erased by EDF ordering.

Do not confuse this with Day 4's EEVDF virtual deadline. SCHED_DEADLINE parameters are a userspace scheduling contract in real-time units. EEVDF deadlines rank eligible fair-class service requests in virtual time.

8. Priority inversion: the scheduler cannot release a lock

Suppose low-priority L holds mutex M, high-priority H blocks on M, and medium-priority M2 preempts L. H is highest priority but cannot run through the critical section; M2 does not need the mutex and can delay the only task able to release it. That is priority inversion.

Priority inheritance changes the owner's **effective** priority to reflect a waiter, possibly through a chain of blocked owners. It does not change the owner's normal policy parameters permanently, make critical sections bounded, or cure deadlock.

`kernel/sched/core.c`, `rt_mutex_setprio()`, lines 7075–7095:

```
/*
 * rt_mutex_setprio - set the current priority of a task
 * @p: task to boost
 * @pi_task: donor task
 *
 * This function changes the 'effective' priority of a task. It does
 * not touch ->normal_prio like __setscheduler().
 *
 * Used by the rt_mutex code to implement priority inheritance
 * logic. Call site only calls if the priority of the task changed.
 */
void rt_mutex_setprio(struct task_struct *p, struct task_struct *pi_task)
{
    int prio, oldprio, queued, running, queue_flag =
        DEQUEUE_SAVE | DEQUEUE_MOVE | DEQUEUE_NOCLOCK;
    const struct sched_class *prev_class, *next_class;
    struct rq_flags rf;
    struct rq *rq;

    /* XXX used to be waiter->prio, not waiter->task->prio */
    prio = __rt_effective_prio(pi_task, p->normal_prio);
```

For a process-shared PI lock, userspace state and the kernel futex PI path connect to an rtmutex. The path queues a proxy waiter and waits on the PI mutex:

`kernel/futex/pi.c`, `futex_lock_pi()`, lines 1014–1033:

```
    raw_spin_lock_irq(&q.pi_state->pi_mutex.wait_lock);
    spin_unlock(q.lock_ptr);
    /*
     * __rt_mutex_start_proxy_lock() unconditionally enqueues the @rt_waiter
     * such that futex_unlock_pi() is guaranteed to observe the waiter when
     * it sees the futex_q::pi_state.
     */
    ret = __rt_mutex_start_proxy_lock(&q.pi_state->pi_mutex, &rt_waiter, current);
    raw_spin_unlock_irq(&q.pi_state->pi_mutex.wait_lock);

    if (ret) {
        if (ret == 1)
            ret = 0;
        goto cleanup;
    }

    if (unlikely(to))
        hrtimer_sleeper_start_expires(to, HRTIMER_MODE_ABS);

    ret = rt_mutex_wait_proxy_lock(&q.pi_state->pi_mutex, to, &rt_waiter);
```

An ordinary pthread mutex does not establish this path. The mutex must request a PI protocol, the platform must support it, and the contention path must actually occur. Even then, PI limits one inversion mechanism; it does not create a worst-case response bound without bounded critical sections, suitable scheduling policies, admitted load, and system-level analysis.

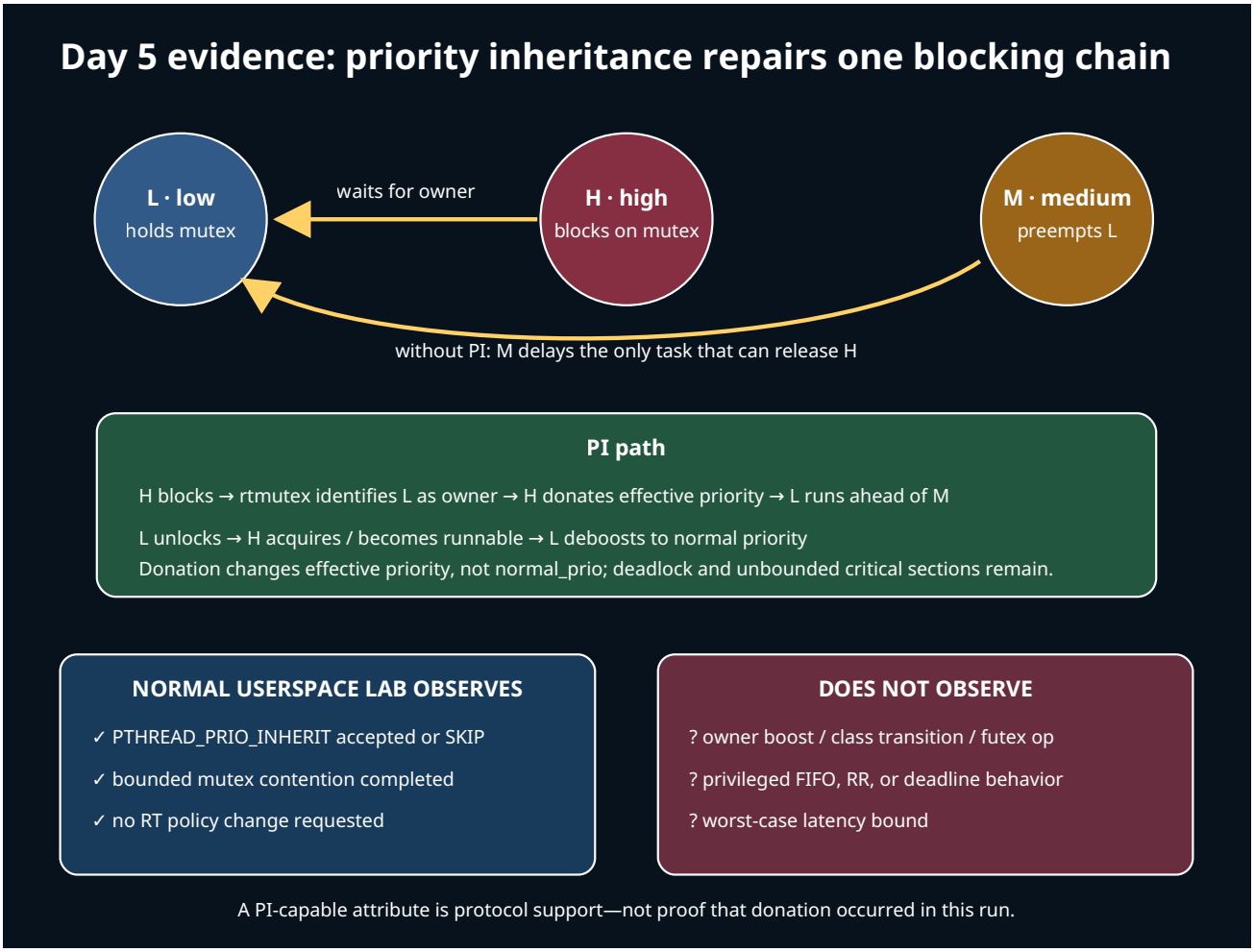
9. Bounded lab: support check, not privileged-policy proof

Run:

```
./15-linux-scheduler-evolution/challenges/day-005-classes-guarantees.sh
```

The existing wrapper builds the project, verifies pinned source anchors, and runs `inversion_lab`. The program:

- requests `PTHREAD_PRIO_INHERIT` in a mutex attribute;
- reports support or `SKIP unsupported`;
- has the main thread hold that mutex while a worker blocks;
- releases it after a bounded sleep and joins the worker;
- explicitly reports `rt_policy_change=none`.



Observed: whether the pthread implementation accepted the PI attribute, whether the bounded contention completed, and that the program requested no RT policy change.

Not observed: an owner priority boost, the exact futex operation, class transition, FIFO/RR behavior, deadline admission, RT throttling, starvation prevention, or any worst-case latency.

Pinned-source proof: the class layout and assertions, RR quantum path, deadline/CBS description, and rtmutex/futex PI connection shown above for Linux 6.12.

Normal pthread scheduling does not reproduce privileged kernel policy. Both threads may remain ordinary fair tasks; a successful PI attribute is capability/protocol evidence, not a demonstration of an RT schedule. The lab intentionally avoids `SCHED_FIFO`, `SCHED_RR`, and `SCHED_DEADLINE`, which commonly require privilege or resource-limit configuration and can disrupt the host.

10. Practical questions and limits

1. If a fair task never runs while a FIFO task spins, which evidence distinguishes intended class precedence from disabled or exhausted RT bandwidth protection?
2. Two RR tasks share a priority. What makes them rotate, and why does that say nothing about service for a lower-class task?
3. A deadline reservation is admitted with runtime 2 ms and period 10 ms. What happens when the task repeatedly demands 4 ms, and why is “deadline class always meets every deadline” false?
4. Why can a high-priority waiter remain blocked even though no runnable task outranks it directly?
5. Which mutex attribute and runtime observation would be needed before hypothesizing a PI futex path? What additional trace would test that hypothesis?
6. Why does PI change effective priority rather than `normal_prio`?
7. Why is the stop class absent from any safe userspace policy experiment?
8. Which limits—affinity, cgroup bandwidth, RT runtime, deadline admission, virtualization, or lock hold time—must be recorded before claiming a response guarantee?

Do not test starvation by launching an unbounded privileged RT loop. Use a disposable, isolated environment with an external watchdog and explicit CPU/bandwidth limits if a later lab is designed for real RT policy. This lesson's wrapper deliberately does not.

Completion prompt

For each class, state one ordering rule and one condition that bounds the claim:

```
stop → deadline → RT/FIFO-RR → fair → idle
```

Then narrate L/H/M2 inversion from mutex ownership through PI donation and deboost. Label every claim as `policy contract`, `Linux 6.12 pinned implementation`, `bounded lab observation`, or `not established`. Treat later upstream as **potentially changed** until checked against a named version and commit.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Classes Guarantees

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

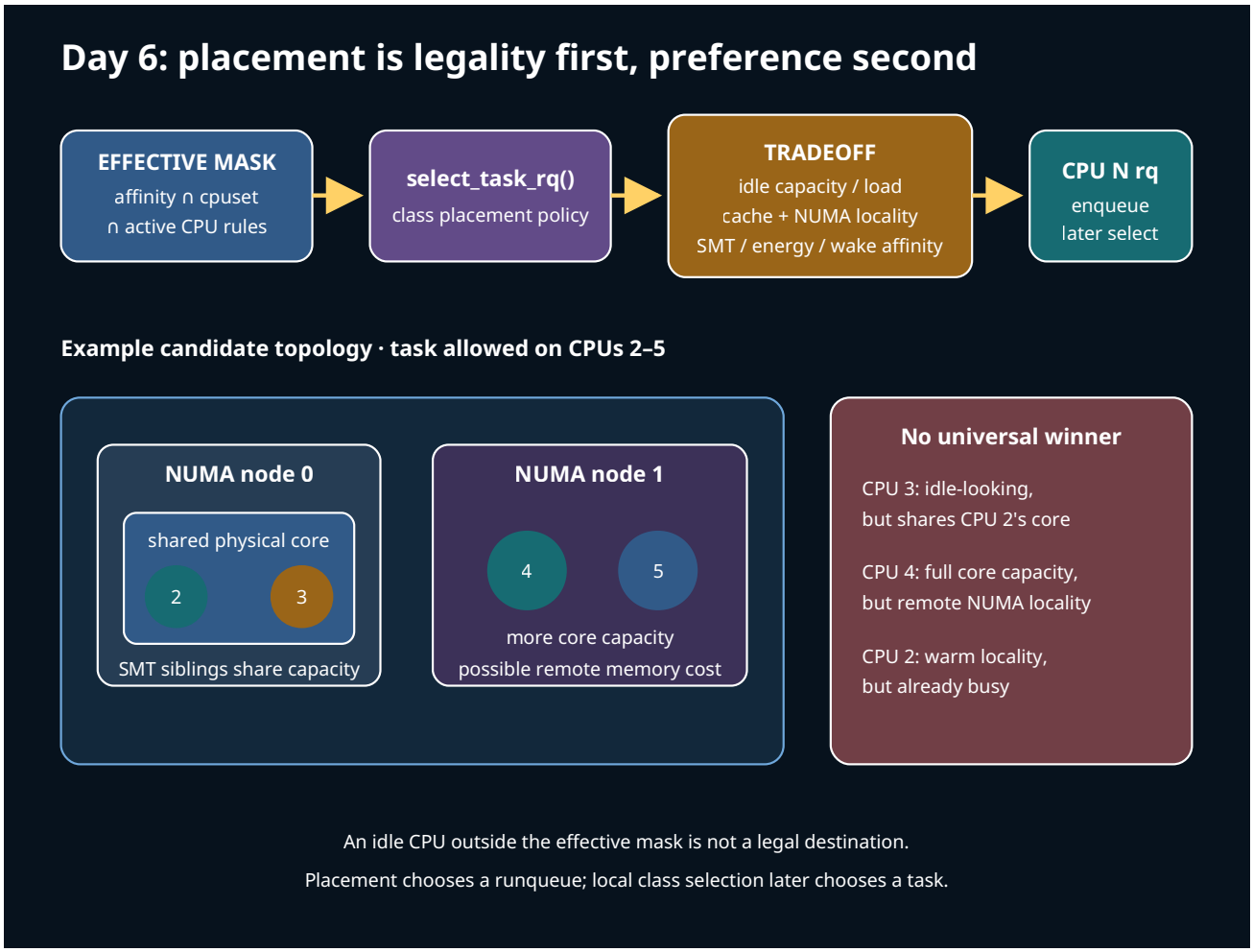
```
bash 15-linux-scheduler-evolution/challenges/day-005-classes-guarantees.sh
```

- View lab source on GitHub

Day 6 — SMP Placement, Topology, and Control

Target time: approximately 3 hours

> Driving question: after a task becomes runnable, which CPUs may run it, which CPU should run it, and what can our evidence actually prove?



Placement is not a second name for selection. Selection chooses a runnable task from a runqueue; placement decides which runqueue should own that task. On one CPU the distinction is easy to miss. On SMP it determines whether idle capacity can be used, whether cache and NUMA locality survive, and whether affinity and administrative isolation remain true.

1. Syntax needed for the placement path

Read CPU masks as sets, not integers. In `cpumask_test_cpu(cpu, p->cpus_ptr)`, `p->cpus_ptr` points at the task's currently effective mask and the result answers membership for one CPU. `cpumask_and(dst, a, b)` computes an intersection. An empty intersection means there is no ordinary legal destination; it does not license the scheduler to choose a convenient CPU outside the constraints.

`for_each_domain(cpu, sd)` is a macro-expanded loop over an RCU-protected hierarchy. The loop body may move from a close topology level toward wider levels. Read the lock and RCU requirements around the call site before treating the macro like an array traversal.

The call

```
new_cpu = p->sched_class->select_task_rq(p, cpu, wake_flags);
```

is an indirect function call through a scheduler-class operation. The common wakeup code sets the protocol; the class supplies policy. A returned CPU number is a placement result, not proof that the task immediately executed there.

`READ_ONCE()` and `WRITE_ONCE()` preserve the shape of an individual compiler access. They do not replace the locks and acquire/release edges that stabilize `cpus_ptr`, `on_rq`, and `on_cpu`. Likewise, `rcu_read_lock()` protects traversal lifetime; it does not freeze load, idle state, or topology against every concurrent change.

2. Linux 0.12: why placement was absent

Use `references/EARLY_LINUX_012_MAP.md`, then read Chapter 8.6 of the local commented Linux 0.12 PDF, `kernel/sched.c`, printed pages 351–372. The map deliberately asks only four base questions: why a task sleeps, where runnable state is recorded, who wakes it, and how another task receives the CPU.

Linux 0.12's `schedule()` scans the fixed `task[]` array for `TASK_RUNNING` tasks and chooses the greatest positive `counter`. If all runnable counters are zero, it recalculates each as half its previous value plus `priority` and scans again. `switch_to(next)` then uses the 80386 hardware task-switch machinery.

This tiny mechanism makes three durable boundaries visible:

1. task state determines eligibility;
2. policy ranks the eligible tasks;
3. the switch transfers execution to the winner.

There is no destination-CPU decision because the implementation is uniprocessor. The single scan is simultaneously the eligible set and the only runqueue. That is exactly why 0.12 is useful here: it shows what SMP had to add. It is not evidence for per-CPU queues, modern affinity, load balancing, cpusets, NUMA, or software context switching.

3. Why the design had to evolve

With multiple CPUs, a single global queue and scan would make unrelated scheduling operations contend on shared state and would ignore where a task's warm cache lines and memory live. Per-CPU runqueues localize the common enqueue, pick, and accounting paths, but they create a new problem: load and runnable work can become uneven.

Modern placement therefore has two different kinds of decision:

- **event placement** chooses a runqueue on wakeup, fork, or exec;
- **load balancing** periodically or reactively moves queued work when runqueues and topology groups are imbalanced.

Neither decision can be reduced to "pick the least loaded CPU." Moving a task consumes cache locality, may turn local NUMA accesses into remote ones, may place two heavy tasks on SMT siblings that share execution resources, and may violate affinity or cpuset rules. Conversely, keeping perfect locality while another package is idle can waste much more capacity than the locality saves.

Historical names must remain separated. Linux 0.12 used a counter scan; the later O(1) scheduler used active and expired priority arrays; classic CFS ranked entities by virtual runtime; pinned Linux 6.12 uses fair scheduling with EEVDF eligibility and virtual deadlines. Those selection eras explain pressure on the design. They do not make an old picker the implementation of current placement.

4. Exact ToyOS boundary

The actual course source in `projects/toyos/src/tasks.c` has two ring-0 kernel tasks. Its private `schedule()` alternates task IDs, refuses a blocked candidate, validates the saved stack and RIP, updates `TASK_RUNNABLE` / `TASK_RUNNING`, and calls `context_switch`. `scheduler_tick()` invokes the same scheduler when a current task exists.

`projects/toyos/src/switch.S` saves `%r15`, `%r14`, `%r13`, `%r12`, `%rbx`, `%rbp`, `%rsp`, and a continuation RIP, loads the next context, and jumps to it. This reconstructs state filter → choose → switch and both cooperative and timer-triggered scheduling.

ToyOS has no CPU identifier, per-CPU `struct rq`, affinity mask, migration, balancing domain, NUMA model, cpuset, hotplug protocol, cgroup CPU controller, NOHZ behavior, KVM vCPU, or userspace return path. It therefore implements none of today's placement subsystems. The absence is a precise boundary, not an invitation to infer them from the round-robin choice.

5. Linux 6.12 pinned source: legality before preference

The source reference for this lesson is commit `adc218676eef25575469234709c2d87185ca223a` in `/home/jd/Documents/linux-6.12`. The following excerpts are verbatim and were checked by normalized, ordered-line matching against that tree.

The fair-class wakeup callback starts with a previous CPU, the waking CPU, and policy flags. The task's PI lock stabilizes the allowed-mask view while it considers fast paths:

`kernel/sched/fair.c`, `select_task_rq_fair()`, lines 8588–8608:

```
static int
select_task_rq_fair(struct task_struct *p, int prev_cpu, int wake_flags)
{
    int sync = (wake_flags & WF_SYNC) && !(current->flags & PF_EXITING);
    struct sched_domain *tmp, *sd = NULL;
    int cpu = smp_processor_id();
    int new_cpu = prev_cpu;
    int want_affine = 0;
    /* SD_flags and WF_flags share the first nibble */
    int sd_flag = wake_flags & 0xF;

    /*
     * required for stable ->cpus_allowed
     */
    lockdep_assert_held(&p->pi_lock);
    if (wake_flags & WF_TTWU) {
        record_wakee(p);

        if ((wake_flags & WF_CURRENT_CPU) &&
            cpumask_test_cpu(cpu, p->cpus_ptr))
            return cpu;
```

Later in this function the fast wakeup path may call `select_idle_sibling()`, while a domain carrying the relevant balance flag can lead to `sched_balance_find_dst_cpu()`. These are preferences among candidates. They do not override the task's effective mask.

The core legality check also accounts for CPU lifecycle and special kernel-thread rules:

`kernel/sched/core.c`, `is_cpu_allowed()`, lines 2383–2407:

```
static inline bool is_cpu_allowed(struct task_struct *p, int cpu)
{
    /* When not in the task's cpumask, no point in looking further. */
    if (!task_allowed_on_cpu(p, cpu))
        return false;

    /* migrate_disabled() must be allowed to finish. */
    if (is_migration_disabled(p))
        return cpu_online(cpu);

    /* Non kernel threads are not allowed during either online or offline. */
    if (!(p->flags & PF_KTHREAD))
        return cpu_active(cpu);

    /* KTHREAD_IS_PER_CPU is always allowed. */
    if (kthread_is_per_cpu(p))
        return cpu_online(cpu);

    /* Regular kernel threads don't get to stay during offline. */
    if (cpu_dying(cpu))
        return false;

    /* But are allowed during online. */
    return cpu_online(cpu);
}
```

This distinction is central:

- the **requested/allowed affinity** is the user's affinity intent, retained where the kernel can preserve it;
- the **effective affinity** is the mask the scheduler can currently enforce after cpuset restrictions, CPU availability, and special migration state are considered.

Do not describe `taskset` output as the entire legality calculation. A task can request CPUs that an enclosing cpuset removes, and a CPU can transition through hotplug states. Conversely, migration-disabled code has a bounded need to finish on its current online CPU. Ordinary userspace does not gain that exception.

Changing affinity and moving a queued task are also distinct. In the pinned core, `__migrate_task()` rechecks `is_cpu_allowed()` because the constraints may have changed again. `move_queued_task()` deactivates under the old runqueue lock, changes the task CPU, locks the destination runqueue, activates there, and checks wakeup preemption. An online destination outside the effective mask is not legal merely because it is idle.

6. Domains encode topology for balancing

Scheduling domains are per-CPU hierarchy views used to compare nearby and progressively wider groups. They are policy structures built from topology and configuration, not a promise that every firmware description is accurate or that the hierarchy never changes.

`kernel/sched/topology.c`, package-topology comment, lines 1136–1159:

```
* Package topology (also see the load-balance blurb in fair.c)
*
* The scheduler builds a tree structure to represent a number of important
* topology features. By default (default_topology[]) these include:
*
*  - Simultaneous multithreading (SMT)
*  - Multi-Core Cache (MC)
*  - Package (PKG)
*
* Where the last one more or less denotes everything up to a NUMA node.
*
* The tree consists of 3 primary data structures:
*
*  sched_domain -> sched_group -> sched_group_capacity
*      ^ ^             ^ ^
*          `-'             `-'
*
* The sched_domains are per-CPU and have a two way link (parent & child) and
* denote the ever growing mask of CPUs belonging to that level of topology.
*
* Each sched_domain has a circular (double) linked list of sched_group's, each
* denoting the domains of the level below (or individual CPUs in case of the
* first domain level). The sched_group linked by a sched_domain includes the
* CPU of that sched_domain [*].
```

An SMT sibling is a logical CPU sharing core resources. Cores may share caches, and cores belong to packages; a NUMA node describes a memory-locality relationship, not simply "the next wider CPU group." Capacity can also be asymmetric. Consequently:

- two idle SMT siblings do not equal two idle physical cores of independent capacity;

- a same-core or same-cache destination may retain more locality;
- a remote NUMA CPU may offer enough free capacity to justify remote memory access;
- placing a task near memory does not prove its pages are actually local.

Load balancing walks domains and groups to decide whether work should move. Wake placement and balancing can arrive at the same CPU, but they run for different events and use different snapshots. A trace of `sched_migrate_task` proves a migration event and its endpoints, not whether wake affinity, periodic balance, hotplug, or an affinity change was the cause.

7. Constraints that alter apparent CPU service

Cpusets and hotplug. A cpuset can narrow where tasks may execute and can partition scheduling domains. CPU hotplug changes online/active/dying state, rebuilds scheduling structures, and may force tasks away. The lab reads its inherited mask; it neither writes cpusets nor onlines or offlines a CPU.

NOHZ and ticks. Tick suppression removes unnecessary periodic ticks on idle or, under the relevant configuration, isolated busy CPUs. It does not suspend scheduling. Wakeups, timers, interrupts, explicit reschedule requests, and remote balancing mechanisms still create work. Sparse tick observations are therefore not proof of a stuck scheduler.

Cgroup CPU weight and quota. In cgroup v2, `cpu.weight` shares available CPU proportionally among competing sibling groups; it is not a reservation when no sibling competes. `cpu.max` imposes bandwidth as quota and period. A runnable task can be affinity-legal and fairly ranked yet receive no service while its group is throttled. Read the task's cgroup membership and the corresponding `cpu.stat`; increasing `nr_throttled` / `throttled_usec` is evidence for bandwidth exhaustion, whereas a weight alone does not prove throttling.

KVM. A guest vCPU executes as a host thread. QEMU/KVM may also have emulator and I/O threads whose placement is separate from vCPU placement. Pinning guest vCPU 0 inside the guest constrains the guest task to that virtual CPU; it does not pin the host vCPU thread, emulator thread, IRQ handling, or memory. Guest steal time reports intervals in which a non-idle vCPU was not supplied by the host, when the platform exposes that accounting. It cannot name the competing host task, distinguish quota from host priority, or prove that an emulator thread was the bottleneck.

These topics interact with placement, but this lesson does not pretend to implement or fully model cpuset partitioning, hotplug state machines, NOHZ housekeeping, cgroup bandwidth, or KVM. It establishes where each can invalidate a simpler placement story.

8. Bounded lab: observation is not implementation proof

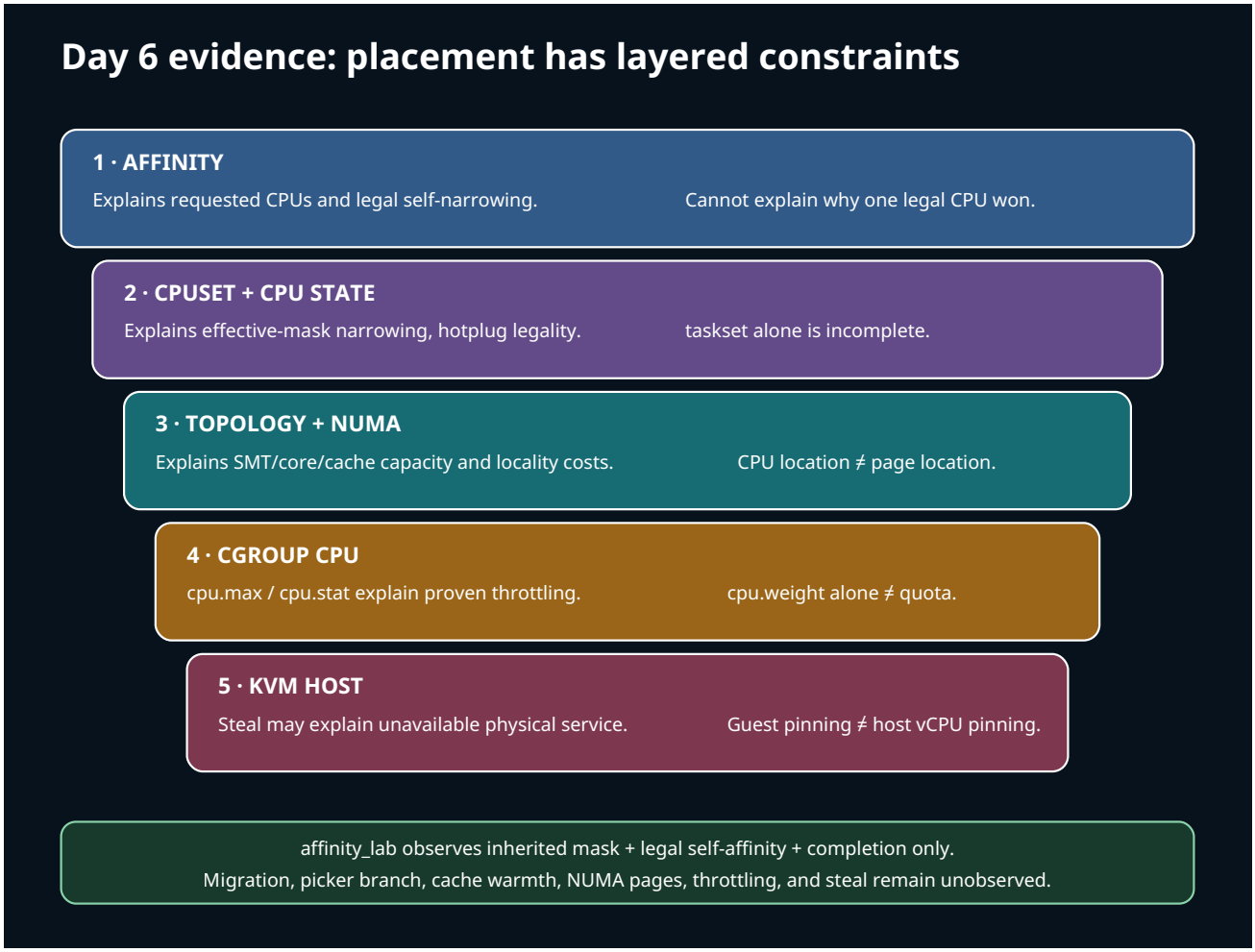
Run the existing wrapper:

```
15-linux-scheduler-evolution/challenges/day-006-smp-placement.sh
```

It runs `make -C projects/linux-scheduler build`, executes `projects/linux-scheduler/scripts/check-source.py`, then runs `build/affinity_lab`. The program calls `sched_getaffinity()` for itself, counts the inherited allowed CPUs, selects only the first CPU

already in that mask, and narrows its own affinity to that CPU. Its expected markers include `allowed_cpus=...`, `first_allowed=...`, `scope=self_only`, and `PASS affinity_lab`; the wrapper then prints `PASS week15-day6`.

This is intentionally safe and small. It does not force CPU 0, alter another process, write cgroups, request RT policy, hotplug a CPU, mount tracefs, or generate sustained load. If the inherited mask contains one CPU, the run still proves legal self-affinity behavior; there is simply no two-CPU migration experiment to perform.



Use this evidence split in the report:

- **Observed by this run:** the host returned an inherited mask; the selected CPU belonged to that mask; narrowing self-affinity succeeded if the PASS marker appeared; the bounded process completed.
- **Not observed by this run:** which `select_task_rq_fair()` branch was taken, whether the task migrated, which domain was traversed, cache warmth, NUMA page location, NOHZ behavior, cgroup throttling, or KVM steal.
- **Proved by pinned source:** at commit `adc218676eef25575469234709c2d87185ca223a`, the quoted functions contain the shown mask, lock, CPU-state, and topology logic in the stated order.
- **Not proved by pinned source:** that the running host uses this exact build, that a particular branch ran during the lab, or that later upstream retains the same details.

Source and observation answer different questions. Never use a matching symbol in the tree as a substitute for a captured event, and never use a userspace timestamp to claim a particular internal branch.

9. Practical placement reasoning

For a delayed runnable task, reason in this order:

1. **Eligibility:** is the task actually runnable rather than sleeping on an I/O or lock predicate?
2. **Legality:** what is its effective CPU mask after affinity, cpuset, hotplug state, and migration constraints?
3. **Class and bandwidth:** is a higher scheduling class running, or is the task's cgroup out of quota?
4. **Placement event:** was this wake placement, fork/exec placement, periodic balancing, forced migration, or merely a sample of the current CPU?
5. **Topology cost:** would moving cross SMT/core/cache/package/NUMA boundaries gain real capacity, and what locality would it lose?
6. **Virtualization:** is the guest runnable while its host vCPU thread is not receiving service?

Consider a worker allowed on CPUs 2–5. CPUs 2 and 3 are SMT siblings; CPUs 4 and 5 are on another NUMA node. CPU 2 is busy, CPU 3 is nominally idle, and CPU 4 has full core capacity. "Choose idle CPU 3" ignores shared core capacity; "choose CPU 4" ignores remote memory; "stay on CPU 2" ignores idle capacity. A defensible answer records the effective mask, workload behavior, topology, page locality evidence, and repeated runtime distribution. It does not promise one universally correct destination.

Mitigations must match evidence. Widen an accidentally narrow cpuset only if placement legality is the constraint. Change `cpu.max` only when throttling evidence increases. Separate vCPU and emulator-thread pinning only after host-side contention is established. Avoid making RT policy, disabling SMT, or turning off NUMA balancing a generic latency prescription.

10. Limits and completion check

Affinity snapshots can race with changes. `/proc` counters aggregate causes. A sampled processor number says where the task was observed, not why it arrived there. Tracepoints add event order but perturb timing and may be unavailable due to configuration, capabilities, lockdown, or tracefs policy. A single fast run proves neither fairness nor a latency bound; a slow run proves neither starvation nor a scheduler defect.

Complete the lesson by narrating:

`runnable event → effective-mask legality → select_task_rq preference → destination runqueue → possible balance/migration → class selection → context switch`.

For each arrow label the evidence as `Linux 0.12 historical`, `ToyOS implemented`, `Linux 6.12 pinned source`, `host observed`, or `hypothesis`. Include one SMT-versus-core tradeoff, one NUMA-locality-versus-capacity tradeoff, one cgroup or KVM alternative explanation, and one result that must be `SKIP` when the relevant interface is unavailable. Any statement about a newer kernel must be separately version checked; otherwise label it `potentially changed`.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Smp Placement

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

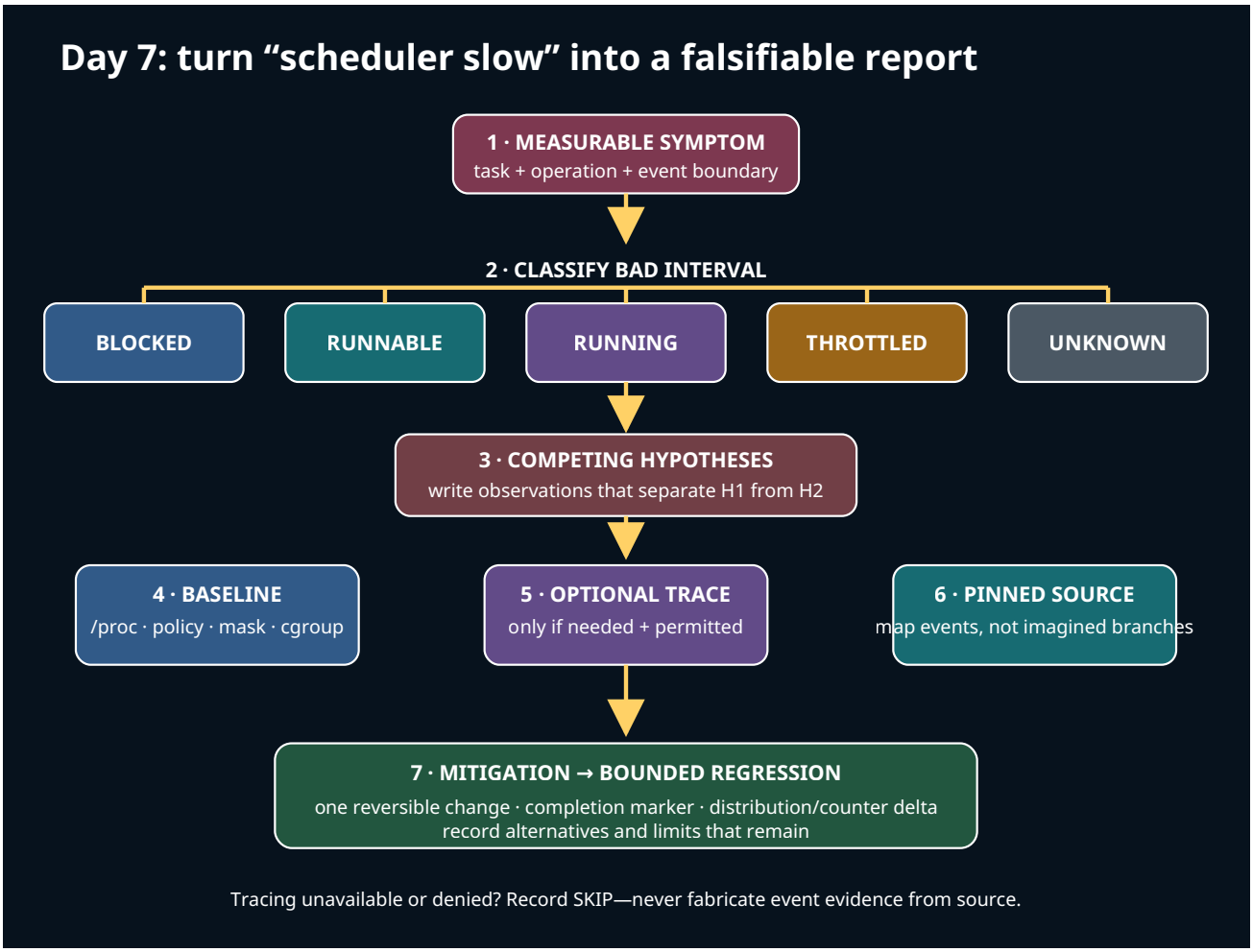
```
bash 15-linux-scheduler-evolution/challenges/day-006-smp-placement.sh
```

- View lab source on GitHub

Day 7 — A Scheduler Debugging Report

Target time: approximately 3 hours

> Driving question: how do we turn “the scheduler is slow” into a bounded, falsifiable report without inventing evidence?



Today is a workflow, not a catalogue of commands. A useful report starts with a visible failure, identifies the scheduler boundary that could produce it, and chooses the least invasive evidence that can separate competing causes. A tracer is optional; a falsifiable hypothesis and an honest evidence boundary are not.

1. Syntax needed to read evidence paths

Kernel trace events are declared with macros. In a `TRACE_EVENT(name, ...)` declaration, `TP_PROTO` describes the generated call signature, `TP_ARGS` passes those values, and `TP_STRUCT__entry` describes recorded fields. This is source structure, not a record of a live event. An event exists in the pinned tree even when the running kernel omits it, tracefs is unavailable, or no one enabled it.

A call through `p->sched_class->select_task_rq(...)` dispatches policy through a function pointer. A direct call such as `trace_sched_switch(...)` is a source landmark. Neither syntax proves that a specific host run reached the call.

When reading `/proc/<pid>/stat`, remember that it is a positional interface. Parenthesized `comm` can contain spaces, so an ad-hoc `cut -d' '` parser is unsafe. Prefer a maintained tool or parse according to `Documentation/filesystems/proc.rst`. In `/proc/<pid>/status`, named lines such as `State`, `Cpus_allowed_list`, `voluntary_ctxt_switches`, and `nonvoluntary_ctxt_switches` are easier to read, but they remain snapshots or cumulative counters.

`READ_ONCE()` is not a timestamp, and a scheduler lock does not make userspace observations atomic with kernel state. Every `ps`, `/proc`, or trace read has a collection interval. The report must say whether it observed a point sample, a cumulative delta, or ordered events.

2. Linux 0.12: a small model for classifying delay

Start with `references/EARLY_LINUX_012_MAP.md` and Chapter 8.6 of the local commented Linux 0.12 PDF, `kernel/sched.c`, printed pages 351–372. Its `schedule()` scans the fixed `task[]` array for `TASK_RUNNING` tasks and chooses the largest positive `counter`. When all runnable counters are zero, it recalculates counters from half the old value plus `priority`, then scans again. `switch_to(next)` transfers execution.

That gives a compact first debugging split:

- if a task is not `TASK_RUNNING`, investigate the event or predicate that should wake it;
- if it is runnable but not selected, investigate ranking and available CPU service;
- if a different task was selected, investigate the switch and subsequent return path.

The old sleep helpers and `wake_up()` make this causal shape visible, but they are not modern tracing semantics. Linux 0.12 is uniprocessor, has no per-CPU runqueues, scheduler classes, cgroup bandwidth, SMP migration, NUMA placement, RT mutex PI chain, or KVM steal accounting. A report may use 0.12 to explain the categories; it may not cite 0.12 as proof of a current host mechanism.

3. Why debugging had to evolve with the scheduler

On SMP, “runnable but delayed” has many non-equivalent causes. The task may be queued behind higher-class work, fairly waiting among peers, constrained to busy CPUs, throttled by cgroup bandwidth, repeatedly migrated, waiting on a priority-inverted owner, or inside a guest whose host vCPU thread is not running. A cumulative context-switch count cannot distinguish these.

Per-CPU runqueues also make CPU identity part of the story. Wake placement and periodic balancing can move work; topology makes two logical CPUs unequal in effective capacity; NUMA adds memory locality costs. Scheduler classes add different contracts. Cgroup controllers and virtualization add periods in which a runnable guest or process receives no physical CPU service for reasons outside fair-queue ranking.

Historical mechanisms must not be collapsed. Linux 0.12's counter scan, the O(1) scheduler, classic CFS, and Linux 6.12 fair EEVDF are different implementations. A modern report maps evidence to the pinned implementation and labels later upstream details `potentially changed` until separately checked.

4. Exact ToyOS boundary

In the actual `projects/toyos/src/tasks.c`, two ring-0 kernel tasks have `TASK_RUNNABLE`, `TASK_RUNNING`, or `TASK_BLOCKED` state. Its private `schedule()` chooses the other task, skips a blocked candidate, validates the saved context, updates states, and calls `context_switch`. `scheduler_tick()` invokes scheduling when a current task exists.

`projects/toyos/src/switch.S` saves the callee-saved registers, `%rsp`, and a continuation RIP, restores the next context, and jumps. Serial markers prove the bounded ToyOS self-tests reached their own completion paths when those tests are run.

ToyOS supplies no `/proc`, scheduler tracepoints, per-CPU queue, affinity, migration, cgroup bandwidth, RT policy, PI mutex chain, NUMA placement, or virtualization. It can teach state → choose → switch. It cannot reproduce any of today's five diagnoses, and its serial output cannot be reported as Linux host tracing.

5. Linux 6.12 pinned source: what a switch event means

All pinned-source claims below refer to commit `adc218676eef25575469234709c2d87185ca223a` in `/home/jd/Documents/linux-6.12`. The excerpts are verbatim and were checked by normalized, ordered-line matching.

In `__schedule()`, selection happens before the switch-only branch:

`kernel/sched/core.c`, `__schedule()`, lines 6647–6662:

```
    next = pick_next_task(rq, prev, &rf);
picked:
    clear_tsk_need_resched(prev);
    clear_preempt_need_resched();
#ifdef CONFIG_SCHED_DEBUG
    rq->last_seen_need_resched_ns = 0;
#endif

    if (likely(prev != next)) {
        rq->nr_switches++;
        /*
         * RCU users of rcu_dereference(rq->curr) may not see
         * changes to task_struct made by pick_next_task().
         */
        RCU_INIT_POINTER(rq->curr, next);
        /*
```

The tracepoint is emitted after accounting and before `context_switch()`:

`kernel/sched/core.c`, `__schedule()`, lines 6684–6697:

```
        ++*switch_count;

        migrate_disable_switch(rq, prev);
        psi_account_irqtime(rq, prev, next);
        psi_sched_switch(prev, next, block);

        trace_sched_switch(preempt, prev, next, prev_state);

        /* Also unlocks the rq: */
        rq = context_switch(rq, prev, next, &rf);
    } else {
        rq_unpin_lock(rq, &rf);
        __balance_callbacks(rq);
        raw_spin_rq_unlock_irq(rq);
```

Therefore, a captured `sched_switch` event can identify the previous and next task at that point. It does not prove that the next task returned to userspace, how long it had been runnable unless paired with suitable wakeup evidence, or why the policy chose it. If `prev == next`, this call site emits no switch event even though scheduling work occurred.

The wakeup tracepoint's own pinned comment is deliberately narrow:

`include/trace/events/sched.h`, `sched_wakeup`, lines 174–180:

```
/*
 * Tracepoint called when the task is actually woken; p->state == TASK_RUNNING.
 * It is not always called from the waking context.
 */
DEFINE_EVENT(sched_wakeup_template, sched_wakeup,
         TP_PROTO(struct task_struct *p),
         TP_ARGS(p));
```

A wakeup-to-switch interval is useful runnable-latency evidence only after correlating the right task and CPU across ordered records. It is not identical to syscall latency, end-to-end request latency, or time blocked on the original predicate.

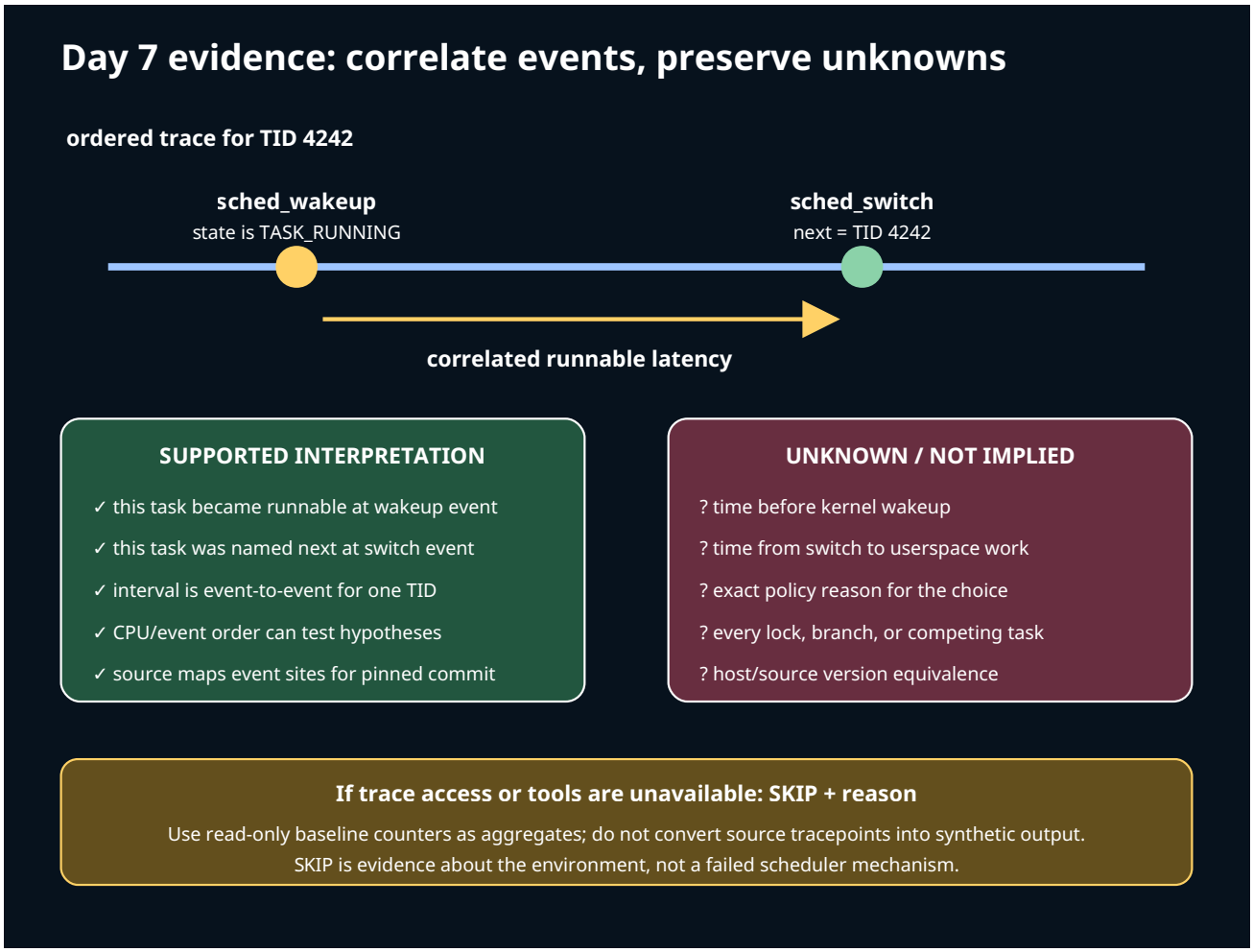
6. Bounded baseline and the proof split

Run the required wrapper:

```
15-linux-scheduler-evolution/challenges/day-007-debug-report.sh
```

It builds `projects/linux-scheduler`, runs `scripts/check-source.py`, and executes `build/report_lab`. The lab prints its PID, reads only `/proc/self/status`, emits the `voluntary_ctxt_switches` and `nonvoluntary_ctxt_switches` lines when available, then prints `report_scope=self_proc_read_only` and `PASS report_lab`. The wrapper ends with `PASS week15-day7`.

The wider project suite also contains bounded wakeup, pipe ping-pong, self-affinity, and mutex-protocol exercises. `make -C projects/linux-scheduler tools` checks for `ps`, `taskset`, `perf`, `trace-cmd`, and `bpftrace`. It runs only the read-only `ps` and `taskset` views. Even when installed, `perf`, `trace-cmd`, and `bpftrace` are deliberately reported as `SKIP ... not run automatically`; tracing requires explicit learner action and host permission. Preserve that SKIP. No trace was captured merely because a tool was present.



Use this prose evidence table for every report:

- **Observation:** the exact command, clock interval, PID/TID, host kernel from `uname`, and literal result. For the core wrapper this is limited to self `/proc` counters and completion markers.
- **Interpretation:** the smallest conclusion supported by the observation. A counter delta means cumulative switches changed; it does not enumerate switch causes.
- **Pinned-source proof:** path, function, exact line range, and commit for the mechanism being compared. Source establishes implementation structure for that commit, not host execution.
- **Unknown:** missing event order, privilege, cgroup ancestry, topology, host-side VM state, or any other fact not collected.
- **SKIP:** the unavailable or intentionally unrun probe and the reason. SKIP is data about the environment, not a failed scheduler mechanism.

7. The debugging workflow

Use the same seven steps for each scenario:

1. State the symptom in user terms: operation, affected task, start/end boundary, and frequency. “The scheduler is slow” is not measurable.
2. Record the environment: host kernel, bare metal or guest, PID/TID, scheduling policy, nice value, effective CPU list, cgroup path, and test duration.

3. Classify the task at the bad interval as blocked, runnable, running, throttled, or unknown. Do not infer state from elapsed time alone.
4. Write two competing hypotheses and one observation that would separate them.
5. Collect the least invasive baseline first; enable event tracing only if baseline evidence cannot answer the question and permission is explicit.
6. Map observed events to the pinned source without claiming unobserved branches.
7. Apply one scoped mitigation, then repeat a bounded regression check with deterministic completion assertions separated from noisy timing distributions.

Useful read-only baselines include:

- `/proc/<pid>/status`: state sample, allowed CPU/memory lists, and cumulative voluntary and nonvoluntary context switches;
- `/proc/<pid>/sched`: policy and per-task scheduler statistics when exposed, noting that fields vary with configuration;
- `/proc/<pid>/schedstat`: cumulative runtime, runqueue delay, and timeslices where enabled; compare deltas over a defined interval rather than raw lifetime totals;
- `/proc/<pid>/stat`: state, user/system CPU time, priority, nice, processor, and policy fields, parsed according to the documented positional format;
- `/proc/stat`: system CPU totals including guest-visible `steal` where applicable;
- `/proc/pressure/cpu`: aggregate CPU pressure, useful for contention context but not attribution to one task;
- `/proc/<pid>/cgroup` plus the matching cgroup v2 `cpu.stat`, `cpu.max`, and `cpu.weight`: membership, throttling counters, quota, and proportional weight.

`ps -L -o pid,tid,stat,cls,rtprio,pri,ni,psr,comm -p <pid>` and `taskset -pc <tid>` are snapshots. They can race with migration and state changes.

8. Scenario A — runnable latency

Symptom. A pipe reader is awakened, but application work begins noticeably later. The boundary of interest is “became runnable” to “was switched in,” not the entire read.

Competing hypotheses. H1: the task was runnable and queued behind other eligible work. H2: the application timestamp began before the kernel wakeup, so most delay was still in the blocked predicate or I/O path.

Evidence table in list form:

- **Baseline:** record repeated end-to-end samples, `/proc/<tid>/schedstat` deltas if available, policy/nice, allowed CPUs, cgroup membership, and CPU pressure over the same bounded interval.
- **Optional event proof:** correlate `sched_wakeup` for the TID with the next `sched_switch` that names it as `next`. `perf sched timehist`, ftrace scheduler events, `trace-cmd record -e sched:sched_wakeup -e sched:sched_switch`, or an equivalent bpftrace script can do this only when installed, supported, permitted, and explicitly run. Otherwise record `SKIP`; this lesson captured none of those traces.
- **Source mapping:** `try_to_wake_up()` and `select_task_rq()` in `kernel/sched/core.c`, `select_task_rq_fair()` in `kernel/sched/fair.c`, then `pick_next_task()` and the quoted `__schedule()` switch site.

- **Mitigation after evidence:** reduce proven CPU contention, repair an overly narrow affinity/cpuset, or shorten the actual blocking critical path. Do not assign RT policy merely because one latency sample was high.
- **Regression:** keep a deterministic completion marker and compare a distribution of wakeup-to-switch intervals under a recorded workload. Use a project-specific budget, not a universal Linux latency threshold.
- **Limit:** `sched_wakeup` to `sched_switch` still excludes later userspace execution and does not identify every policy input.

9. Scenario B — cgroup throttling mistaken for unfairness

Symptom. A CPU-bound service runs in bursts followed by pauses even though it remains runnable and an allowed CPU appears available.

Competing hypotheses. H1: its cgroup exhausted `cpu.max` and was throttled until the next period. H2: ordinary competition or host unavailability explains the pauses.

Evidence table in list form:

- **Baseline:** resolve `/proc/<pid>/cgroup` to the correct cgroup v2 directory; record `cpu.max`, `cpu.weight`, and before/after `cpu.stat`. Increasing `nr_throttled` and `throttled_usec` over the incident interval supports H1. A low `cpu.weight` alone does not prove quota throttling.
- **Optional event proof:** cgroup and scheduler tracing can improve timing correlation, but event names and availability vary. If unavailable or not permitted, use the controller counters and mark tracing `SKIP`.
- **Source mapping:** pinned `kernel/sched/fair.c` contains CFS bandwidth accounting; `kernel/sched/core.c` exports cgroup CPU statistics and the `cpu.max`/`cpu.weight` interfaces. `Documentation/admin-guide/cgroup-v2.rst` defines their user contract.
- **Mitigation after evidence:** right-size quota/period, reduce workload, or move the service to the intended cgroup. Changing nice cannot create runtime while the group is throttled.
- **Regression:** under the same bounded load, check completion, throughput distribution, and `cpu.stat` deltas. Record whether reduced throttling came from more budget or less work.
- **Limit:** aggregate cgroup counters do not attribute every throttled interval to one thread, and a container may hide the host hierarchy.

10. Scenario C — migration and locality

Symptom. Throughput or tail latency worsens after deployment, and sampled `psr` values suggest that a worker moves among CPUs.

Competing hypotheses. H1: migrations lose cache or NUMA locality. H2: movement is a helpful response to imbalance, while memory placement or another resource is the actual bottleneck.

Evidence table in list form:

- **Baseline:** record `Cpus_allowed_list`, `Mems_allowed_list`, `taskset -pc`, topology from the read-only system interfaces available to the learner, and repeated CPU/runtime samples. A changing `psr` sample suggests movement but is not an ordered migration log.
- **Optional event proof:** `sched:sched_migrate_task` records origin and destination when enabled. Pair it with switch events and, if available, NUMA/page evidence. `perf`, ftrace, `trace-cmd`, or bpftrace absence/access denial is an honest SKIP.
- **Source mapping:** `select_task_rq_fair()` in `kernel/sched/fair.c`, migration helpers in `kernel/sched/core.c`, and domain construction in `kernel/sched/topology.c`. A migration event does not itself identify which path caused it.
- **Mitigation after evidence:** adjust an erroneous affinity/cpuset, colocate threads and memory, or widen placement if imbalance costs more than locality. Pinning everything can create the overload it was meant to cure.
- **Regression:** compare throughput and latency distributions while recording migration count, allowed mask, and NUMA environment. Revert if reduced migrations reduce capacity.
- **Limit:** CPU placement does not reveal page placement; NUMA locality and scheduler capacity need separate evidence.

11. Scenario D — RT starvation and priority inheritance

Symptom. A normal-policy thread receives little service, or a high-priority thread waits behind a lock held by lower-priority work.

Competing hypotheses. H1: runnable RT work legitimately precedes fair-class work or consumes its configured bandwidth. H2: the high-priority thread is blocked on a mutex and the owner is delayed; PI may or may not be configured and active.

Evidence table in list form:

- **Baseline:** record per-thread policy/class, `rtprio`, state, and the synchronization boundary. Distinguish `R` from a blocked state. Read relevant RT bandwidth settings only if the environment permits; do not change them during diagnosis.
- **Lab boundary:** `inversion_lab` requests `PTHREAD_PRIO_INHERIT` and reports `pi_mutex=requested_supported` or `pi_mutex=SKIP unsupported`, but it makes no RT policy change. Protocol support under normal policy does not prove a kernel PI boost occurred or establish an RT response bound.
- **Optional event proof:** when available and enabled, `sched_pi_setprio` can show a priority change. Scheduler switches show service order. Lock-specific tools may be required to establish ownership. Without these, label PI execution unknown.
- **Source mapping:** class ordering and `__schedule()` are in `kernel/sched/core.c`; RT queueing is in `kernel/sched/rt.c`; PI behavior is implemented through the rtmutex paths and exposed by `sched_pi_setprio` in `include/trace/events/sched.h`.
- **Mitigation after evidence:** shorten the critical section, use a correctly supported PI mutex where the scheduling design requires it, bound RT utilization, or repair priorities. Never prescribe FIFO as a generic latency cure.

- **Regression:** assert lock completion and absence of deadlock, then measure bounded waiter latency under a declared policy/load matrix. Keep watchdogs and restore any privileged policy in a controlled external test.
- **Limit:** `ps` priorities are samples; a PI-capable mutex attribute alone is not an observed donation chain.

12. Scenario E — guest vCPU steal

Symptom. A task inside a VM is runnable and guest CPU pressure rises, yet wall-clock progress stalls.

Competing hypotheses. H1: the host is not scheduling the guest's vCPU thread, visible as increasing guest steal time. H2: the guest scheduler, cgroup quota, I/O emulation, or an emulator thread is the bottleneck.

Evidence table in list form:

- **Guest baseline:** record `/proc/stat` steal deltas over the incident window, per-task runtime/runqueue-delay deltas, effective affinity, CPU pressure, and whether the kernel reports virtualization. Zero steal does not prove uninterrupted physical service on every platform.
- **Host evidence:** if the learner controls the host, identify the QEMU vCPU and emulator threads, their affinity, cgroup, policy, and service. Guest-only evidence cannot name a host competitor. If host access is unavailable, write `SKIP host-side attribution`.
- **Optional event proof:** host scheduler traces may correlate vCPU-thread wakeups and switches. Guest traces describe scheduling within the guest. Keep the two timelines separate and account for clock alignment. This lesson captured neither.
- **Source mapping:** `Documentation/virt/kvm/x86/msr.rst` defines KVM steal accounting; host scheduling still reaches the pinned core scheduler paths. Emulator-thread delay is not proven by guest steal alone.
- **Mitigation after evidence:** reduce host oversubscription, correct host vCPU/emulator affinity, or adjust a proven host cgroup budget. Guest task pinning does not pin its host vCPU thread.
- **Regression:** compare bounded guest work, steal deltas, and host vCPU service under the same allocation. Preserve host-side SKIP when unavailable.
- **Limit:** steal time is unavailable service, not a root-cause label.

13. Writing the final report

Keep the report compact but auditable:

1. **Symptom and boundary:** what was delayed, which TID, and between which two events?
2. **Environment:** kernel release, source commit used for comparison, CPU mask, policy, cgroup, topology/VM status, duration, and clock.
3. **Hypotheses:** at least two causes with predictions that differ.
4. **Evidence ledger:** each item labeled observation, pinned-source proof, interpretation, unknown, or SKIP.
5. **Causal path:** blocked/runnable state → placement → runqueue/class → bandwidth or host availability → switch → return. Mark every unsupported arrow.

6. **Mitigation:** one reversible change tied to the strongest evidence.
7. **Regression:** deterministic completion plus a bounded distribution/counter delta.
8. **Limits:** perturbation, races, aggregation, permissions, source/host version mismatch, and alternatives still consistent with the data.

Contradictory evidence stays in the report. If `cpu.stat` does not change while latency spikes, throttling becomes less likely for that interval; do not hide the result. If a tracer is denied, do not mount tracefs, change lockdown, or claim events from source.

14. Practical limits and completion check

Tracing perturbs scheduling and can overflow buffers. `/proc` reads race with state changes. Per-task counters are cumulative and may have configuration-dependent semantics. Clock domains can differ across guest and host. A tracepoint near a function does not expose every lock, branch, virtual deadline, or reason code. The running host may not be the pinned 6.12 tree.

Complete the day by choosing one of the five scenarios and producing the report structure above. The core wrapper's actual results may be cited only for self `/proc` counters and completion. Any optional `perf`, ftrace, `trace-cmd`, or bpftrace section must contain literal captured output from an explicitly run command or the word `SKIP`; source excerpts are never synthetic trace output.

Finish with one claim labeled `Linux 6.12 pinned source`, one labeled `host observed`, one `unknown`, and one `potentially changed`. That separation—not the number of tools used—is the debugging skill.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Debug Report

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 15-linux-scheduler-evolution/challenges/day-007-debug-report.sh
```

- View lab source on GitHub